

LABORATORY MANUAL

PCCSL307 DATA STURUCTURES LAB

THIRD SEMESTER

COMPUTER SCIENCE AND ENGINEERING

KTU SYLLABUS – 2024 ONWARDS

	Prepared by	Verified by	Approved by
Signature			
Date			

INSTITUTION VISION

-To be a centre of excellence in academics and research for moulding students to become competent engineering professionals with innovative ideas and dedicated to the upliftment of the deprived socio-economic sections of the society through effective teaching learning process.

INSTITUTION MISSION

-To transform budding engineers into employable quality professionals. -To inculcate the need of pursuing higher education among the graduate students so as to create a society of highly educated professionals. -To impart ethical values and social consciousness among the students so as to create responsible and socially committed engineers. -To create an excellent academic ambience which encourages creative thinking, innovations and research.

VISION OF COMPUTER SCIENCE AND ENGINEERING DEPARTMENT

To be a premier department nurturing students to become academically, socially and industrially competent computer professionals and researchers contributing innovations for the betterment of the society.

MISSION OF COMPUTER SCIENCE AND ENGINEERING DEPARTMENT

M1: To invigorate academic environment by providing an exposure to advanced technologies with an emphasis on basic principles of computer science and engineering.

M2: To enhance leadership, problem solving and entrepreneurship skills and inculcate social responsibilities and ethical values in students.

M3: To encourage students for pursuing higher education, research and innovations.

PROGRAM OUTCOME

PO No.	PO Statement
PO1	Engineering Knowledge:Apply knowledge of mathematics, natural science, computing and engineering fundamentals, and an engineering specialization as specified in WK1 to WK4 respectively to develop solutions to complex engineering problems.
PO2	Problem Analysis:Identify, formulate, research literature and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences and engineering sciences with holistic considerations for sustainable development.
PO3	Design/development of solutions:Design creative solutions for complex engineering problems and design systems, components or processes to meet identified needs with appropriate consideration for public health and safety, whole-life cost, net zero carbon as well as resource, cultural, societal, and environmental considerations as required.
PO4	Investigation:Conduct investigations of complex engineering problems using research methods including research based knowledge, design of experiments, analysis and interpretation of data, and synthesis of information to provide valid conclusions.
PO5	Tool Usage:Create, select and apply, and recognize limitations of appropriate techniques, resources, and modern engineering and IT tools, including prediction and modelling, to complex engineering problems.
PO6	The Engineer and the World:When solving complex engineering problems, analyze and evaluate sustainable development impacts* to: society, the economy, sustainability, health and safety, legal frameworks, and the environment.
PO7	Ethics:Apply ethical principles and commit to professional ethics and norms of engineering practice and adhere to relevant national and international laws. Demonstrate an understanding of the need for diversity and inclusion.
PO8	Individual and Collaborative Team work:Function effectively as an individual, and as a member or leader in diverse and inclusive teams and in multi-disciplinary, face-to-face, remote and distributed settings.
PO9	Communication:Communicate effectively and inclusively on complex engineering activities with the engineering community and with society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations, taking into account cultural, language, and learning differences.
PO10	Project Management and Finance:Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.
PO11	Lifelong learning: Recognize the need for, and have the preparation and ability for

	i. independent and life-long learning ii. adaptability to new and emerging technologies and iii. critical thinking in the broadest context of technological change

Programme Specific Outcomes(PSO's)

- PSO 1 : Our graduates will analyse, design and solve problems of varying complexity in the fields of Computer Science and Engineering.

- PSO 2 : Our graduates will function responsibly and ethically, fully committed to profession and society.

- PSO 3 : Our graduates will have the ability to apply their knowledge and skills to succeed in their career and productively engage in innovations and higher learning.

Program Educational Objectives(PEO's)

- PEO 1 : Acquire knowledge in subject domain and apply it together with hardware and software skill in the design and implementation of systems which are adaptable to the industrial trends.

- PEO 2 : Socially committed with moral and ethical values in bringing socially acceptable solutions to real world problems for society through innovations and entrepreneurial activities and enhancing the interpersonal, leadership and professional skills to lead a successful career and life.

- PEO 3 : Actively engage in continuous learning process adapting to the current trends by engaging in lifelong learning, thus pursuing higher studies and research in the subject domain.

COURSE OUTCOME

CO	Description Bloom's taxonomy level	Bloom's Knowledge Level (KL)
CO1	Model a real world problem using suitable data structure and implement the solution.	Applying(P)
CO2	Compare efficiency of different data structures in terms of time and space complexity.	Analyzing(A)
CO3	Evaluate the time complexities of various searching and sorting algorithms.	Evaluate(E)
CO4	Differentiate static and dynamic data structures in terms of their advantages and application.	Applying(P)

CO- PO Mapping

CO	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2	PSO 3
PCCSL 307.1	3	3	2	1	3	-	-	3	-	-	3	3	-	2
CCSL 307.2	2	3	1	2	3	-	-	3	-	-	3	3	2	-
CCSL 307.3	2	3	1	1	2	2	2	3	-	-	3	3	-	2
CCSL 307.4	2	2	2	-	2	-	-	3	2	2	3	2	-	3
	2.25	2.75	1.50	1.33	2.50	2	2	3	2	2	3	2.75	2	2.33

DATA STRUCTURES LAB

(Common to CS/CA/CM/CD/CR/AI/AM/AD/CB/CN/CC/CU/CI/CG)

Course Code	**PCCSL307**	**CIE Marks**	50
Teaching Hours/Week (L: T:P: R)	0:0:3:0	**ESE Marks**	50
Credits	2	**Exam Hours**	2 Hrs. 30 Min.
Prerequisites (if any)	GYEST204	**Course Type**	Lab

Course Objectives:

To give practical experience for learners on implementing different linear and non linear data structures, and algorithms for searching and sorting.

Expt. No.	**Experiments**
1	Find the sum of two sparse polynomials using arrays
2	Find the transpose of a sparse matrix and sum of two sparse matrices.
3	Convert infix expression to postfix (or prefix) and then evaluate using stack,
4	Implement Queue, DEQUEUE, and Circular Queue using arrays.
5	Implement backward and forward navigation of visited web pages in a web browser (i.e. back and forward buttons) using doubly linked list operations.
6	Implement addition and multiplication of polynomials using singly linked lists.
7	Create a binary tree for a given simple arithmetic expression and find the prefix / postfix equivalent.
8	Implement a dictionary of word-meaning pairs using binary search trees.
9	Find the shortest distance of every cell from a landmine inside a maze.
10	We have three containers whose sizes are 10 litres, 7 litres, and 4 litres, respectively. The 7-litre and 4-litre containers start out full of water, but the 10-litre container is initially empty. We are allowed one type of operation: pouring the contents of one container into another, stopping only when the

	source container is empty or the destination container is full. We want to know if there is a sequence of pourings that leaves exactly 2 litres in the 7 or 4-litre container. Model this as a graph problem and solve.
11	Implement the find and replace feature in a text editor.
12	Given an array of sorted items, implement an efficient algorithm to search for specific items in the array.
13	Implement Bubble sort, Insertion Sort, Radix sort, Quick Sort, and Merge Sort and compare the number of steps involved.
14	The General post office wishes to give preferential treatment to its customers. They have identified the customer categories as Defence personnel, Differently abled, Senior citizen, Ordinary. The customers are to be given preference in the decreasing order - Differently abled, Senior citizen, Defence personnel, Normal person. Generate the possible sequence of completion.
15	Implement a spell checker using a hash table to store a dictionary of words for fast lookup. Implement functions to check if a word is valid and to suggest corrections for misspelled words.
16	Simulation of a basic memory allocator and garbage collector using doubly linked list
17	The CSE dept is organizing a tech fest with so many exciting events. By participating in an event, you can claim for activity points as stipulated by KTU. Each event i gives you A[i] activity points where A is an array. If you are not allowed to participate in more than k events, what's the max number of points that you can earn?
18	Merge K sorted lists into a single sorted list using a heap. Use a min-heap to keep track of the smallest element from each list. Repeatedly extract the smallest element and insert the next element from the corresponding list into the heap until all lists are merged.

Course Assessment Method

(CIE: 50 marks, ESE: 50 marks)

Continuous Internal Evaluation Marks (CIE):

Attendance	**Preparation/Pre-Lab Work experiments, Viva and Timely completion of Lab Reports / Record (Continuous Assessment)**	**Internal Examination**	**Total**
5	25	20	50

End Semester Examination Marks (ESE):

Procedure/ Preparatory work/ Design/ Algorithm	Conduct of experiment/ Execution of work/ troubleshooting/ Programming	Result with valid inference/ Quality of Output	Viva voce	Record	Total
10	15	10	10	5	50

- Submission of Record: Students shall be allowed for the end semester examination only upon submitting the duly certified record.
- Endorsement by External Examiner: The external examiner shall endorse the record

Course Outcomes (COs)

At the end of the course students should be able to:

Course Outcome		Bloom's Knowledge Level (KL)
CO1	Model a real world problem using suitable data structure and implement the solution.	**K3**
CO2	Compare efficiency of different data structures in terms of time and space complexity.	**K4**
CO3	Evaluate the time complexities of various searching and sorting algorithms.	**K5**
CO4	Differentiate static and dynamic data structures in terms of their advantages and application.	**K3**

Note*: K1- Remember, K2- Understand, K3- Apply, K4- Analyse, K5- Evaluate, K6- Create*

CO- PO Mapping (Mapping of Course Outcomes with Program Outcomes)

	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12
CO1	3	3	3	3				3				3
CO2	3	3	3	3				3				3
CO3	3	3	3	3				3				3
CO4	3	3	3	3				3				3

1: Slight (Low), 2: Moderate (Medium), 3: Substantial (High), -: No Correlation

Text Books				
Sl. No	**Title of the Book**	**Name of the Author/s**	**Name of the Publisher**	**Edition and Year**
1	Fundamentals of Data Structures in C	Ellis Horowitz, Sartaj Sahni and Susan Anderson-Freed,	Universities Press,	2/e, 2007
2	Introduction to Algorithms	Thomas H Cormen, Charles Leisesrson, Ronald L Rivest, Clifford Stein	PHI	3/e, 2009

Reference Books				
Sl. No	**Title of the Book**	**Name of the Author/s**	**Name of the Publisher**	**Edition and Year**
1	Classic Data Structures	Samanta D.	Prentice Hall India.	2/e, 2018
2	Data Structures and Algorithms	Aho A. V., J. E. Hopcroft and J. D. Ullman	Pearson Publication.	1/e, 2003
3	Introduction to Data Structures with Applications	Tremblay J. P., P. G. Sorenson	Tata McGraw Hill.	2/e, 2017
4	Theory and Problems of Data Structures	Lipschutz S.	Schaum's Series	2/e, 2014

Video Links (NPTEL, SWAYAM…)	
No	**Link ID**
1	https://nptel.ac.in/courses/106102064
2	https://ocw.mit.edu/courses/6-851-advanced-data-structures-spring-2012/

ContContinuous Assessment (25 Marks)

1. **Preparation and Pre-Lab Work (7 Marks)**
    - Pre-Lab Assignments: Assessment of pre-lab assignments or quizzes that test understanding of the upcoming experiment.
    - Understanding of Theory: Evaluation based on students' preparation and understanding of the theoretical background related to the experiments.

2. **Conduct of Experiments (7 Marks)**
    - Procedure and Execution: Adherence to correct procedures, accurate execution of experiments, and following safety protocols.
    - Skill Proficiency: Proficiency in handling equipment, accuracy in observations, and troubleshooting skills during the experiments.
    - Teamwork: Collaboration and participation in group experiments.
2. **Lab Reports and Record Keeping (6 Marks)**
    - Quality of Reports: Clarity, completeness and accuracy of lab reports. Proper documentation of experiments, data analysis and conclusions.
    - Timely Submission: Adhering to deadlines for submitting lab reports/rough record and maintaining a well-organized fair record.
4. **Viva Voce (5 Marks)**
    - Oral Examination: Ability to explain the experiment, results and underlying principles during a viva voce session.

Final Marks Averaging: The final marks for preparation, conduct of experiments, viva, and record are the average of all the specified experiments in the syllabus.

Evaluation Pattern for End Semester Examination (50 Marks)

1. **Procedure/Preliminary Work/Design/Algorithm (10 Marks)**
    - Procedure Understanding and Description: Clarity in explaining the procedure and understanding each step involved.
    - Preliminary Work and Planning: Thoroughness in planning and organizing materials/equipment.
    - Algorithm Development: Correctness and efficiency of the algorithm related to the experiment.
    - Creativity and logic in algorithm or experimental design.
2. **Conduct of Experiment/Execution of Work/Programming (15 Marks)**
    - Setup and Execution: Proper setup and accurate execution of the experiment or programming task.
2. **Result with Valid Inference/Quality of Output (10 Marks)**
    - Accuracy of Results: Precision and correctness of the obtained results.
    - Analysis and Interpretation: Validity of inferences drawn from the experiment or quality of program output.

4. **Viva Voce (10 Marks)**
    - Ability to explain the experiment, procedure results and answer related questions
    - Proficiency in answering questions related to theoretical and practical aspects of the subject.
5. **Record (5 Marks)**
    - Completeness, clarity, and accuracy of the lab record submitted.

INDEX

SL. NO.	NAME OF THE EXPERIMENT	PAGE NO.
1	Find the sum of two sparse polynomials using arrays	1
2	Find the transpose of a sparse matrix and sum of two sparse matrices.	5
3	Convert infix expression to postfix (or prefix) and then evaluate using stack,	10
4	Implement Queue, DEQUEUE, and Circular Queue using arrays.	13
5	Implement backward and forward navigation of visited web pages in a web browser (i.e. back and forward buttons) using doubly linked list operations.	28
6	Implement addition and multiplication of polynomials using singly linked lists.	33
7	Create a binary tree for a given simple arithmetic expression and find the prefix / postfix equivalent.	43
8	Implement a dictionary of word-meaning pairs using binary search trees.	49
9	Find the shortest distance of every cell from a landmine inside a maze.	54
10	We have three containers whose sizes are 10 litres, 7 litres, and 4 litres, respectively. The 7-litre and 4-litre containers start out full of water, but the 10-litre container is initially empty. We are allowed one type of operation: pouring the contents of one container into another, stopping only when the source container is empty or the destination container is full. We want to know if there is a sequence of pourings that leaves exactly 2 litres in the 7 or 4-litre container. Model this as a graph problem and solve.	58
11	Implement the find and replace feature in a text editor.	63
12	Given an array of sorted items, implement an efficient algorithm to search for specific items in the array.	65
13	Implement Bubble sort, Insertion Sort, Radix sort, Quick Sort, and Merge Sort and compare the number of steps involved.	67
14	The General post office wishes to give preferential treatment to its customers. They have identified the customer categories as Defence personnel, Differently abled, Senior citizen, Ordinary. The customers are to be given preference in the decreasing order - Differently abled, Senior citizen, Defence personnel, Normal person. Generate the possible sequence of completion.	80
15	Implement a spell checker using a hash table to store a dictionary of words for fast lookup. Implement functions to check if a word is valid and to suggest corrections for misspelled words.	86

16	Simulation of a basic memory allocator and garbage collector using doubly linked list	89
17	The CSE dept is organizing a tech fest with so many exciting events. By participating in an event, you can claim for activity points as stipulated by KTU. Each event i gives you A[i] activity points where A is an array. If you are not allowed to participate in more than k events, what's the max number of points that you can earn?	94
18	Merge K sorted lists into a single sorted list using a heap. Use a min-heap to keep track of the smallest element from each list. Repeatedly extract the smallest element and insert the next element from the corresponding list into the heap until all lists are merged.	96

1. SUM OF TWO SPARSE POLYNOMIALS USING ARRAYS.

Aim:

Find the sum of two sparse polynomials using arrays.

Algorithm:

Step 1: Start

Step 2: Define Structure

Define a structure Term with two integer members:

- coeff → coefficient of the term
- exp → exponent of the term

Step 3: Read First Polynomial

1. Ask the user to enter the number of non-zero terms.
2. For each term:
    - Read the coefficient and exponent.
3. Store the terms in an array poly1.
4. Return the number of terms n1.

Step 4: Read Second Polynomial

1. Ask the user to enter the number of non-zero terms.
2. For each term:
    - Read the coefficient and exponent.
3. Store the terms in an array poly2.
4. Return the number of terms n2.

Step 5: Add the Two Polynomials

1. Initialize three indices i, j, and k to zero.
2. Compare terms of both polynomials based on exponents:
    - If the exponent of poly1[i] is greater, copy it to result[k].

- If the exponent of poly2[j] is greater, copy it to result[k].
- If both exponents are equal:
  - Add coefficients.
  - If the sum is not zero, store the result term.

3. Increment indices accordingly.
4. Copy any remaining terms from either polynomial to the result array.
5. Return the number of terms in the polynomial.

Step 6: Display the Polynomials

1. Print each polynomial term in the form ax^b.
2. Insert + or - symbols appropriately between terms.
3. If the polynomial is empty, print 0.

Step 7: Display the Result

Print:

- First polynomial
- Second polynomial
- Sum of the two polynomials

Step 8: Stop

Program:

```
#include <stdio.h>
#include <stdlib.h>
#define MAX_TERMS 100
typedef struct {
   int coeff;
   int exp;
} Term;

int read_poly(Term poly[]) {
   int num_terms, i;
   printf("Enter the number of non-zero terms: ");
   scanf("%d", &num_terms);
   printf("Enter coefficient and exponent for each term (e.g., 2 3 for 2x^3):\n");
   for (i = 0; i < num_terms; i++) {
      scanf("%d %d", &poly[i].coeff, &poly[i].exp);
```

```
    }
    return num_terms;
}

int add_poly(Term poly1[], int n1, Term poly2[], int n2, Term result[]) {
    int i = 0, j = 0, k = 0;
    while (i < n1 && j < n2) {
        if (poly1[i].exp > poly2[j].exp) {
            result[k++] = poly1[i++];
        } else if (poly2[j].exp > poly1[i].exp) {
            result[k++] = poly2[j++];
        } else {
            int sum_coeff = poly1[i].coeff + poly2[j].coeff;
            if (sum_coeff != 0) {
                result[k].coeff = sum_coeff;
                result[k].exp = poly1[i].exp;
                k++;
            }
            i++;
            j++;
        }
    }
    while (i < n1) {
        result[k++] = poly1[i++];
    }

    while (j < n2) {
        result[k++] = poly2[j++];
    }

    return k;
}
void print_poly(Term poly[], int n) {
    if (n == 0) {
        printf("0\n");
        return;
    }
    for (int i = 0; i < n; i++) {
        printf("%dx^%d", poly[i].coeff, poly[i].exp);
        if (i < n - 1 && poly[i+1].coeff > 0) {
            printf(" + ");
        } else if (i < n - 1 && poly[i+1].coeff < 0) {
            printf(" "); // Print a space if the next coeff is negative, the - will handle the
sign
        }
    }
    printf("\n");
}

int main() {
```

```
    Term poly1[MAX_TERMS], poly2[MAX_TERMS], result[MAX_TERMS];
    int n1, n2, n_result;

    printf("--- First Polynomial ---\n");
    n1 = read_poly(poly1);

    printf("--- Second Polynomial ---\n");
    n2 = read_poly(poly2);

    n_result = add_poly(poly1, n1, poly2, n2, result);

    printf("\nFirst polynomial: ");
    print_poly(poly1, n1);
    printf("Second polynomial: ");
    print_poly(poly2, n2);
    printf("Sum polynomial: ");
    print_poly(result, n_result);

    return 0;
}
```

Output

```
--- First Polynomial ---
Enter the number of non-zero terms: 2
Enter coefficient and exponent for each term (e.g., 2 3 for 2x^3):
2 5
5 3
--- Second Polynomial ---
Enter the number of non-zero terms: 2
Enter coefficient and exponent for each term (e.g., 2 3 for 2x^3):
4 5
3 3

First polynomial: 2x^5 + 5x^3
Second polynomial: 4x^5 + 3x^3
Sum polynomial: 6x^5 + 8x^3
```

2. TRANSPOSE OF A SPARSE MATRIX AND SUM OF TWO SPARSE MATRICES.

Aim:

Find the transpose of a sparse matrix and sum of two sparse matrices.

Algorithm:

Step 1: Start.

Step 2: Display menu:

- Transpose
- Addition
- Exit

Step 3: Read user choice.

Step 4: If choice is:

- 1 → Call transpose()
- 2 → Call sum()
- 3 → Stop the program
- Else → Display "Invalid Choice".

Step 5: Repeat until Exit is selected.

Step 6 :Stop.

Algorithm: Transpose of Sparse Matrix

Step 1: Start.

Step 2: Read number of rows, columns, and non-zero elements.

Step 3: Store sparse matrix in 3-tuple form.

Step 4: Interchange row and column values for each non-zero element.

Step5 : Store result as transpose sparse matrix.

Step 6: Display original sparse matrix.

Step 7: Display transpose sparse matrix.

Step 8: Stop.

Algorithm: Addition of Two Sparse Matrices

Step 1: Start.

Step 2: Read rows, columns, and non-zero elements of the first matrix.

Step 3: Read rows, columns, and non-zero elements of the second matrix.

Step 4: Check if matrix dimensions are equal.

- If not equal, display error and stop.

Step5 : Compare positions of non-zero elements:

- If positions are the same, add values.
- Else copy a smaller position element.

Step 6: Copy remaining elements if any.

Step 7: Display the sum sparse matrix.

Step 8: Stop.

Program:

```
#include<stdio.h>
void transpose();
void sum();
void main()
{
   int c;
   printf("1.Transpose\n2.Addition\n3.Exit\n");
   while(1)
   {
      printf("Enter choice:");
      scanf("%d",&c);
      switch(c)
      {
         case 1:
            transpose();
            break;
         case 2:
            sum();
            break;
         case 3:
            return;
         default:
            printf("Invalid Choice");
      }

   }
```

```
}
void transpose()
{
   int s[5][3],t[5][3],rows,cols,nonZero;
   printf("Enter the NO. of raws,columns and Non Zero elements of Matrix: ");
   scanf("%d%d%d",&rows,&cols,&nonZero);
   s[0][0] = rows;
   s[0][1] = cols;
   s[0][2] = nonZero;
   printf("Enter row, column and value for each non-zero element:\n");
   for (int i = 1; i <= nonZero; i++) {
      scanf("%d %d %d", &s[i][0], &s[i][1], &s[i][2]);
   }
   t[0][0] = cols;
   t[0][1] = rows;
   t[0][2] = nonZero;

   for (int i = 1; i <= nonZero; i++) {
      t[i][0] = s[i][1];
      t[i][1] = s[i][0];
      t[i][2] = s[i][2];
   }
  printf("\nSparse Matrix:\n");
   printf("Row Col Val\n");
   for (int i = 0; i <= nonZero; i++) {
      printf("%d   %d   %d\n", s[i][0], s[i][1], s[i][2]);
   }
   printf("\nTranspose of Sparse Matrix:\n");
   printf("Row Col Val\n");
   for (int i = 0; i <= nonZero; i++) {
      printf("%d   %d   %d\n", t[i][0], t[i][1], t[i][2]);
 }
}
void sum()
{
   int a[5][3], b[5][3], sum[5][3];
   int i = 1, j = 1, k = 1;
   printf("Enter rows, columns and non-zero elements of first matrix: ");
   scanf("%d %d %d", &a[0][0], &a[0][1], &a[0][2]);

   printf("Enter row, column and value:\n");
   for (i = 1; i <= a[0][2]; i++) {
      scanf("%d %d %d", &a[i][0], &a[i][1], &a[i][2]);
   }
   printf("Enter rows, columns and non-zero elements of second matrix: ");
   scanf("%d %d %d", &b[0][0], &b[0][1], &b[0][2]);

   printf("Enter row, column and value:\n");
   for (i = 1; i <= b[0][2]; i++) {
      scanf("%d %d %d", &b[i][0], &b[i][1], &b[i][2]);
```

```
    }

    if (a[0][0] != b[0][0] || a[0][1] != b[0][1]) {
       printf("Matrix addition not possible!\n");
       return;
    }
    sum[0][0] = a[0][0];
    sum[0][1] = a[0][1];
    i = j = 1;
    while (i <= a[0][2] && j <= b[0][2]) {
       if (a[i][0] == b[j][0] && a[i][1] == b[j][1]) {
          sum[k][0] = a[i][0];
          sum[k][1] = a[i][1];
          sum[k][2] = a[i][2] + b[j][2];
          i++; j++; k++;
       }
       else if (a[i][0] < b[j][0] ||(a[i][0] == b[j][0] && a[i][1] < b[j][1])) {
          sum[k][0] = a[i][0];
          sum[k][1] = a[i][1];
          sum[k][2] = a[i][2];
          i++; k++;
       }
       else {
          sum[k][0] = b[j][0];
          sum[k][1] = b[j][1];
          sum[k][2] = b[j][2];
          j++; k++;
       }
    }
    while (i <= a[0][2]) {
       sum[k][0] = a[i][0];
       sum[k][1] = a[i][1];
       sum[k][2] = a[i][2];
       i++; k++;
    }
    while (j <= b[0][2]) {
       sum[k][0] = b[j][0];
       sum[k][1] = b[j][1];
       sum[k][2] = b[j][2];
       j++; k++;
    }
    sum[0][2] = k - 1;
    printf("\nSum of Sparse Matrices:\n");
    printf("Row Col Val\n");
    for (i = 0; i <= sum[0][2]; i++) {
       printf("%d   %d   %d\n", sum[i][0], sum[i][1], sum[i][2]);
    }
}
```

Output

```
1.Transpose
2.Addition
3.Exit
Enter choice:1
Enter the NO. of raws,columns and Non Zero elements of Matrix: 3 3 3
Enter row, column and value for each non-zero element:
0 1 3
1 2 5
2 0 6

Sparse Matrix:
Row Col Val
3   3   3
0   1   3
1   2   5
2   0   6

Transpose of Sparse Matrix:
Row Col Val
3   3   3
1   0   3
2   1   5
0   2   6
Enter choice:2
Enter rows, columns and non-zero elements of first matrix: 3 3 3
Enter row, column and value:
0 0 2
1 1 3
2 2 6
Enter rows, columns and non-zero elements of second matrix: 3 3 3
Enter row, column and value:
0 0 1
1 1 2
2 2 3

Sum of Sparse Matrices:
Row Col Val
3   3   3
0   0   3
1   1   5
2   2   9
Enter choice:3
Exiting!!!!!!!
```

3. CONVERT INFIX EXPRESSION TO POSTFIX (OR PREFIX) AND THEN EVALUATE USING STACK.

Aim:

Convert infix expression to postfix (or prefix) and then evaluate using stack.

Algorithm:

Step 1: Start

Step 2: Read Infix Expression

- Read the infix expression from the user.

Step 3: Convert Infix to Postfix

1. Initialize an empty stack for operators
2. Scan the infix expression from left to right.
3. If the symbol is an operand (digit), add it to the postfix expression.
4. If the symbol is '(', push it onto the stack.
5. If the symbol is ')', pop operators from stack and add to postfix until '(' is found.
6. If the symbol is an operator:
    - Pop operators from stack while they have higher or equal priority.
    - Push the current operator onto the stack.
7. After scanning the expression, pop all remaining operators from the stack and add to postfix.

Step 4: Evaluate Postfix Expression.

1. Initialize an empty stack for integers.
2. Scan the postfix expression from left to right.
3. If the symbol is an operand, push it onto the stack.
4. If the symbol is an operator:
    - Pop two operands from the stack.
    - Perform the operation.
    - Push the result back onto the stack.
5. After evaluation, the top of the stack contains the final result.

Step 5: Display Result

- Print the postfix expression.
- Print the evaluated result.

Step 6: Stop

Program:

```
#include <stdio.h>
#include <ctype.h>
```

```
#include <string.h>
#include <math.h>
char charStack[100];
int topChar = -1;

void pushChar(char x) {
charStack[++topChar] = x;
}

char popChar() {
if (topChar == -1)
return -1;
else
return charStack[topChar--];
}

int priority(char x) {
if (x == '(')
return 0;
if (x == '+' || x == '-')
return 1;
if (x == '*' || x == '/')
return 2;

if(x == '^')
return 3;
return 0;
}

int intStack[100];
int topInt = -1;

void pushInt(int x) {
intStack[++topInt] = x;
}

int popInt() {
return intStack[topInt--];
}

int main() {
char infix[100], postfix[100], *e, x;
int k = 0;
printf("Enter the infix expression: ");
scanf("%s", infix);
e = infix;
while (*e != '\0') {
if (isdigit(*e)) {
postfix[k++] = *e;
} else if (*e == '(') {
```

```
pushChar(*e);
} else if (*e == ')') {
while ((x = popChar()) != '(')
postfix[k++] = x;
} else {
while (priority(charStack[topChar]) >= priority(*e))
postfix[k++] = popChar();
pushChar(*e);
}
e++;
}
while (topChar != -1) {
postfix[k++] = popChar();
}
postfix[k] = '\0';

printf("\nPostfix Expression: %s\n", postfix);

e = postfix;
int n1, n2, n3, num;
while (*e != '\0') {
if (isdigit(*e)) {

num = *e - '0';
pushInt(num);
} else {
n1 = popInt();
n2 = popInt();
switch (*e) {
case '+': n3 = n2 + n1; break;
case '-': n3 = n2 - n1; break;
case '*': n3 = n2 * n1; break;
case '/': n3 = n2 / n1; break;
case '^': n3=pow(n2,n1);
break;
}
pushInt(n3);
}
e++;
}
printf("\nResult of the expression = %d\n", popInt());
return 0;
}
```

Output

```
Enter infix expression: (1+(2*3))/7
Postfix expression: 123*+7/
Result = 1
```

4. IMPLEMENT QUEUE, DEQUEUE, AND CIRCULAR QUEUE

Aim:

Implement Queue, Dequeue, and Circular Queue using arrays.

a. QUEUE IMPLEMENTATION

Algorithm:

Step 1: Start

Step 2: Initialize Queue

1. Define an integer array queue of size SIZE.
2. Initialize front = -1 and rear = -1.

Step 3: Display Menu

Display the following menu repeatedly:

1. Enqueue
2. Dequeue
3. Display
4. Exit

Step 4: Read User Choice

Read the user's choice and perform the corresponding operation.

Step 5: Enqueue Operation

1. Read the value to be inserted.
2. Check if rear == SIZE - 1:
    - If true, print Queue Overflow.
3. Otherwise:
    - Increment rear.
    - Insert the value into queue[rear].

Step 6: Dequeue Operation

1. Check if front == rear:
    - If true, print Queue Underflow.
2. Otherwise:
    - Increment front.
    - Display the deleted element queue[front].

Step 7: Display Operation

1. Check if front == rear:
    - If true, print Queue is Empty.

2. Otherwise:
    - Display all elements from queue[front + 1] to queue[rear].

Step 8: Exit

If the user selects Exit:

1. Display exit message.
   Terminate the program.

Step 9: Stop

Program:

```
#include <stdio.h>
#define SIZE 100
int queue[SIZE];
int front = -1, rear = -1;
void enqueue()
{
int value;
printf("Enter value to enqueue: ");
scanf("%d", &value);
if (rear == SIZE - 1)
printf("Queue Overflow\n");
else
{
rear++;
queue[rear] = value;
}
}
void dequeue()
{ if (front == rear)
printf("Queue Underflow\n");
else
{ front++;
printf("Dequeued %d\n", queue[front]);
}
}
void display()
{ if (front==rear)
printf("Queue is Empty\n");
else
{ printf("Queue elements: ");
for (int i = front+1; i <= rear; i++)
printf("%d ", queue[i]);
printf("\n");
}
}
int main()
```

```
{
int choice, value;
printf("\n*** MENU ***\n");
printf("1. Enqueue\n");
printf("2. Dequeue\n");
printf("3. Display\n");
printf("4. Exit\n");
while (1)
{
printf("Enter your choice: ");
scanf("%d", &choice);
switch (choice) {
case 1:
enqueue();
break;
case 2:
dequeue();
break;
case 3:
display();
break;
case 4:
printf("Exiting program.\n");
return 0;
default:
printf("Invalid choice! Please try again.\n");
}
}
return 0;
}
```

Output:

```
*** MENU ***
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 22
Enter your choice: 1
Enter value to enqueue: 45
Enter your choice: 1
Enter value to enqueue: 25
Enter your choice: 1
Enter value to enqueue: 35
Enter your choice: 1
Enter value to enqueue: 56
Enter your choice: 1
```

Enter value to enqueue: 1
Queue Overflow
Enter your choice: 3
Queue elements: 22 45 25 35 56
Enter your choice: 2
Dequeued 22
Enter your choice: 2
Dequeued 45
Enter your choice: 2
Dequeued 25
Enter your choice: 2
Dequeued 35
Enter your choice: 2
Dequeued 56
Enter your choice: 2
Queue Underflow
Enter your choice: 4
Exiting program.

b. DEQUE IMPLIMENTATION

Algorithm:

Step 1: Start

Step 2: Initialize

- Define a constant CAPACITY = 5
- Declare an array arr[CAPACITY]
  Initialize:
  - front = 0
  - rear = 0
  - size = 0

Step 3: Define isFull()

- If size == CAPACITY
  - Return true
- Else
  - Return false

Step 4: Define isEmpty()

- If size == 0
  - Return true
- Else
  - Return false

Step 5: Insert at Front

1. Check if deque is full using isFull()
    - If full, display "Deque is full" and return
2. Update front = (front − 1 + CAPACITY) % CAPACITY
3. Insert the element at arr[front]
4. Increment size

Step 6: Insert at Rear

1. Check if deque is full
    - If full, display "Deque is full" and return
2. Insert the element at arr[rear]
3. Update rear = (rear + 1) % CAPACITY
4. Increment size

Step 7: Delete from Front

1. Check if deque is empty using isEmpty()
    - If empty, display "Deque is empty" and return
2. Update front = (front + 1) % CAPACITY
3. Decrement size

Step 8: Delete from Rear

1. Check if deque is empty
    - If empty, display "Deque is empty" and return
2. Update rear = (rear − 1 + CAPACITY) % CAPACITY
3. Decrement size

Step 9: Display Deque

1. Check if deque is empty.
    - If empty, display "Deque is empty".
2. Else:
    - Start from front
    - Print all size elements using circular traversal.

Step 10: Main Menu

1. Display menu options:
    - Insert at Front
    - Insert at Rear
    - Delete from Front
    - Delete from Rear
    - Display
    - Exit
2. Read user choice.
3. Perform the corresponding operation using switch.
4. Repeat until the user selects Exit.

Step 11: Stop.

Program:

```
#include <stdio.h>
#include<stdlib.h>
#define CAPACITY 5
int arr[CAPACITY];
int front = 0;
int rear = 0;
int size = 0;
int isFull() {
return size == CAPACITY;
}
int isEmpty() {
return size == 0;
}
void insertFront(int value) {
if (isFull()) {
printf("Deque is full\n");
return;
}
front = (front - 1 + CAPACITY) % CAPACITY;
arr[front] = value;

size++;
}
void insertRear(int value) {
if (isFull()) {
printf("Deque is full\n");
return;
}
arr[rear] = value;
rear = (rear + 1) % CAPACITY;
size++;
}
void deleteFront() {
if (isEmpty()) {
printf("Deque is empty\n");
return;
}
front = (front + 1) % CAPACITY;
size--;
}
void deleteRear() {
if (isEmpty()) {
printf("Deque is empty\n");
return;
}
rear = (rear - 1 + CAPACITY) % CAPACITY;
size--;
}
void displayDeque() {
```

```
if (isEmpty()) {
printf("Deque is empty\n");
return;
}
printf("Deque contents: ");
int i = front;
for (int count = 0; count < size; count++) {
printf("%d ", arr[i]);
i = (i + 1) % CAPACITY;
}
printf("\n");
}

// Main menu
int main() {
int choice, item;
printf("\n--- Deque Menu ---\n");
printf("1. Insert at Front\n");
printf("2. Insert at Rear\n");
printf("3. Delete from Front\n");
printf("4. Delete from Rear\n");
printf("5. Display\n");
printf("6. Exit\n");
while (1) {
printf("Enter your choice: ");
scanf("%d", &choice);
switch (choice) {
case 1:
printf("Enter item to insert at front: ");
scanf("%d", &item);
insertFront(item);
break;
case 2:
printf("Enter item to insert at rear: ");
scanf("%d", &item);
insertRear(item);
break;
case 3:
deleteFront();
break;
case 4:
deleteRear();
break;
case 5:
displayDeque();
break;
case 6:
printf("Exiting program.\n");
exit(0);
default:
```

```
printf("Invalid choice. Try again.\n");
}
}
return 0;
}
```

Output:

```
--- Deque Menu ---
1. Insert at Front
2. Insert at Rear
3. Delete from Front
4. Delete from Rear
5. Display
6. Exit
Enter your choice: 1
Enter item to insert at front: 10

--- Deque Menu ---
1. Insert at Front
2. Insert at Rear
3. Delete from Front
4. Delete from Rear
5. Display
6. Exit
Enter your choice: 1
Enter item to insert at front: 20

--- Deque Menu ---
1. Insert at Front
2. Insert at Rear
3. Delete from Front
4. Delete from Rear
5. Display
6. Exit
Enter your choice: 1
Enter item to insert at front: 30

--- Deque Menu ---
1. Insert at Front
2. Insert at Rear
3. Delete from Front
4. Delete from Rear
5. Display
6. Exit
Enter your choice: 2
Enter item to insert at rear: 40

--- Deque Menu ---
```

1. Insert at Front
2. Insert at Rear
3. Delete from Front
4. Delete from Rear
5. Display
6. Exit

Enter your choice:
2
Enter item to insert at rear:
50

--- Deque Menu ---
1. Insert at Front
2. Insert at Rear
3. Delete from Front
4. Delete from Rear
5. Display
6. Exit
Enter your choice: 2
Enter item to insert at rear: 60
Deque is full

--- Deque Menu ---
1. Insert at Front
2. Insert at Rear
3. Delete from Front
4. Delete from Rear
5. Display
6. Exit
Enter your choice: 5
Deque contents: 30 20 10 40 50

--- Deque Menu ---
1. Insert at Front
2. Insert at Rear
3. Delete from Front
4. Delete from Rear
5. Display
6. Exit
Enter your choice: 3

--- Deque Menu ---
1. Insert at Front
2. Insert at Rear
3. Delete from Front
4. Delete from Rear
5. Display
6. Exit
Enter your choice: 5
Deque contents: 20 10 40 50

--- Deque Menu ---
1. Insert at Front
2. Insert at Rear
3. Delete from Front
4. Delete from Rear
5. Display
6. Exit
Enter your choice: 4

--- Deque Menu ---
1. Insert at Front
2. Insert at Rear
3. Delete from Front
4. Delete from Rear
5. Display
6. Exit
Enter your choice: 5
Deque contents: 20 10 40

--- Deque Menu ---
1. Insert at Front
2. Insert at Rear
3. Delete from Front
4. Delete from Rear
5. Display
6. Exit
Enter your choice:
6
Exiting program.

c. CIRCULAR QUEUE IMPLEMENTATION
Algorithm:

Step 1: Start

Step 2: Initialize

- Define constant SIZE = 5
- Declare array queue[SIZE]
- Initialize:
    - front = -1
    - rear = -1

Step 3: Enqueue Operation

1. Check if the queue is full
    - If (rear + 1) % SIZE == front, display "Queue Overflow" and stop.

2. If the queue is empty (front == -1 and rear == -1):
    - Set front = 0 and rear = 0
    - Insert the element at queue[rear]
3. Else:
    - Update rear = (rear + 1) % SIZE
    - Insert the element at queue[rear]

Step 4: Dequeue Operation

1. Check if the queue is empty
    - If front == -1 and rear == -1, display "Queue Underflow" and stop.
2. If there is only one element (front == rear):
    - Display the dequeued element
    - Set front = -1 and rear = -1
3. Else:
    - Display the dequeued element
    - Update front = (front + 1) % SIZE

Step 5: Display Operation

1. Check if the queue is empty
    - If empty, display "Queue is Empty"
2. Else:
    - Start from front
    - Print elements until rear is reached using circular traversal

Step 6: Main Menu

1. Display menu options:
    - Enqueue
    - Dequeue
    - Display
    - Exit
2. Read user choice
3. Perform selected operation using switch
4. Repeat until Exit is chosen

Step 7: Stop

Program:

```
#include <stdio.h>
#define SIZE 5
int queue[SIZE];
int front = -1, rear = -1;

void enqueue(int item) {
if ((rear + 1) % SIZE == front) {
printf("Queue Overflow\n");
```

```
} else if (front == -1 && rear == -1) {
front = rear = 0;
queue[rear] = item;
} else {
rear = (rear + 1) % SIZE;
queue[rear] = item;
}
}

void dequeue() {
if (front == -1 && rear == -1) {
printf("Queue Underflow\n");
} else if (front == rear) {
printf("Dequeued %d\n", queue[front]);
front = rear = -1;
} else {
printf("Dequeued %d\n", queue[front]);
front = (front + 1) % SIZE;
}
}

void display() {
if (front == -1 && rear == -1) {
printf("Queue is Empty\n");
return;
}
printf("Queue elements: ");
int i = front;
while (1) {
printf("%d ", queue[i]);
if (i == rear)
break;

i = (i + 1) % SIZE;
}
printf("\n");
}

int main() {
int choice, value;
while (1) {
printf("\n*** MENU ***\n");
printf("1. Enqueue\n");
printf("2. Dequeue\n");
printf("3. Display\n");
printf("4. Exit\n");
printf("Enter your choice: ");
scanf("%d", &choice);
switch (choice) {
case 1:
```

```
printf("Enter value to enqueue: ");
scanf("%d", &value);
enqueue(value);
break;
case 2:
dequeue();
break;
case 3:
display();
break;
case 4:
printf("Exiting program.\n");
return 0;
default:
printf("Invalid choice! Please try again.\n");
}
}
return 0;
}
```

Output:

```
*** MENU ***
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 10

*** MENU ***
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 20

*** MENU ***
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 30

*** MENU ***
1. Enqueue
2. Dequeue
3. Display
```

```
4. Exit
Enter your choice: 1
Enter value to enqueue: 40

*** MENU ***
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 50

*** MENU ***
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 60
Queue Overflow

*** MENU ***
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 3
Queue elements: 10 20 30 40 50

*** MENU ***
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 2
Dequeued 10

*** MENU ***
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 2
Dequeued 20

*** MENU ***
1. Enqueue
2. Dequeue
3. Display
4. Exit
```

```
Enter your choice: 3
Queue elements: 30 40 50

*** MENU ***
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 60

*** MENU ***
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter value to enqueue: 70

*** MENU ***
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 3
Queue elements: 30 40 50 60 70

*** MENU ***
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 4
Exiting program.
```

5. IMPLEMENT BACKWARD AND FORWARD NAVIGATION OF VISITED WEB PAGES IN A WEB BROWSER USING DOUBLY LINKED LIST OPERATIONS

Aim:

To implement backward and forward navigation of visited web pages in a web browser using doubly linked list operations.

Algorithm:

Step 1: Start

Step 2: Define a doubly linked list node with fields:

- url (string)
- prev (pointer to previous node)
- next (pointer to next node)

Initialize current pointer to NULL.

Step 3: Display the menu with the following options repeatedly:

1. Visit New Page
2. Go Back
3. Go Forward
4. Display Current Page
5. Exit

Step 4: Read the user's choice and perform the corresponding operation:

- If choice = 1 (Visit New Page):
  a. Read the new URL.
  b. Delete all forward nodes from current->next.
  c. Create a new node and assign the URL.
  d. Link the new node after the current.
  e. Move the current pointer to the new node.
  f. Display the visited URL.
- If choice = 2 (Go Back):
  a. If there is a previous node, move current to current->prev and display the page.
  b. Otherwise, display a message that there is no previous page.
- If choice = 3 (Go Forward):
  a. If there is a next node, move current to current->next and display the page.
  b. Otherwise, display a message that there is no next page.
- If choice = 4 (Display Current Page):
  a. If current is not NULL, display current->url.
  b. Otherwise, display that no page has been visited yet.
- If choice = 5 (Exit):
  a. Free all allocated nodes by traversing from the head of the list.
  b. Display exit message.

Step 5: Repeat Step 3 until the user chooses Exit.

Step 6: Stop

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
struct Node {
char url[100];
struct Node* prev;
struct Node* next;
};

struct Node* current = NULL;
void visitNewPage(char url[]) {
        struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
        strcpy(newNode->url, url);
        newNode->prev = current;
        newNode->next = NULL;

        if (current != NULL) {
        current->next = newNode;
        }

        current = newNode;
        printf("Visited: %s\n", current->url);
}

void goBack() {
        if (current == NULL || current->prev == NULL) {
        printf("Cannot go back. No previous page.\n");
        } else {
        current = current->prev;
        printf("Moved back to: %s\n", current->url);
        }
}

void goForward() {
        if (current == NULL || current->next == NULL) {
        printf("Cannot go forward. No next page.\n");
        } else {
        current = current->next;
        printf("Moved forward to: %s\n", current->url);
        }
}

void displayCurrentPage() {
        if (current == NULL) {
```

```
        printf("No page visited yet.\n");
        } else {
        printf("Current Page: %s\n", current->url);
        }
}

int main() {
        int choice;
        char url[100];
        do {
        printf("\nMenu\n----\n");
        printf("1. Visit New Page\n");
        printf("2. Go Back\n");
        printf("3. Go Forward\n");
        printf("4. Display Current Page\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        getchar();
        switch(choice) {
        case 1:
                printf("Enter URL to visit: ");
                fgets(url,100,stdin);
                visitNewPage(url);
                break;
        case 2:
                goBack();
                break;
        case 3:
                goForward();
                break;
        case 4:
                displayCurrentPage();
                break;
        case 5:
                printf("Exiting Browser Simulation.\n");
                break;
        default:
                printf("Invalid choice.\n");
        }
        } while(choice != 5);

        return 0;
}
```

Output:

```
Menu ----
1. Visit New Page
2. Go Back
3. Go Forward
4. Display Current Page
5. Exit
Enter your choice: 1
Enter URL to visit: google.com
Visited: google.com

Menu ----
1. Visit New Page
2. Go Back
3. Go Forward
4. Display Current Page
5. Exit
Enter your choice: 2
Cannot go back. No previous page.
Menu ----
1. Visit New Page
2. Go Back
3. Go Forward
4. Display Current Page
5. Exit
Enter your choice: 3
Cannot go forward. No next page.
Menu ----
1. Visit New Page
2. Go Back
3. Go Forward
4. Display Current Page
5. Exit
Enter your choice: 4
Current Page: google.com
Menu ----
1. Visit New Page
2. Go Back
3. Go Forward
4. Display Current Page
5. Exit
Enter your choice: 1
Enter URL to visit: amazon.com
Visited: amazon.com
Menu ----
1. Visit New Page
2. Go Back
3. Go Forward
4. Display Current Page
```

```
5. Exit
Enter your choice: 2
Moved back to: google.com
Menu ----
1. Visit New Page
2. Go Back
3. Go Forward
4. Display Current Page
5. Exit
Enter your choice: 3
Moved forward to: amazon.com
Menu ----
1. Visit New Page
2. Go Back
3. Go Forward
4. Display Current Page
5. Exit
Enter your choice: 4
Current Page: amazon.com
Menu ----
1. Visit New Page
2. Go Back
3. Go Forward
4. Display Current Page
5. Exit
Enter your choice: 5
Exiting Browser Simulation.
```

6. IMPLEMENT ADDITION AND MULTIPLICATION OF POLYNOMIALS USING SINGLY LINKED LISTS.

a. IMPLEMENT ADDITION OF POLYNOMIALS USING SINGLY LINKED LISTS.

Aim:

To implement addition of polynomials using singly linked lists.

Algorithm:

Step 1: Start

Step 2: Define the Node Structure

Each node of the linked list represents a term of the polynomial and contains:

- coeff → coefficient of the term
- exp → exponent of the term
- next → pointer to the next node

Step 3: Create a New Node

- Allocate memory for a new node.
- Store the given coefficient and exponent.
- Set the next pointer to NULL.
- Return the newly created node.

Step 4: Insert a Node at the End of the Polynomial

- If the polynomial list is empty, make the new node the head.
- Otherwise, traverse the list to the last node.
- Attach the new node at the end.
- Return the updated head pointer.

Step 5: Display the Polynomial

- If the list is empty, print 0.
- Otherwise, traverse the list from head to end:
    - Print each term in the form coeffx^exp.
    - Print " + " between terms.
- Move to the next node until the end.

Step 6: Add Two Polynomials

Assume both polynomials are arranged in descending order of exponents.

1. Initialize the result polynomial as empty.

2. Traverse both polynomials simultaneously:
    - If the exponent of the first polynomial term is greater:
        - Copy that term to the result.
        - Move to the next term in the first polynomial.
    - If the exponent of the second polynomial term is greater:
        - Copy that term to the result.
        - Move to the next term in the second polynomial.
    - If both exponents are equal:
        - Add the coefficients.
        - If the sum is not zero, insert the resulting term into the result.
        - Move to the next terms of both polynomials.
3. If any terms remain in the first polynomial, copy them to the result.
4. If any terms remain in the second polynomial, copy them to the result.

Step 7: Input Polynomials

- Read the number of terms in the first polynomial.
- Read coefficient and exponent for each term and insert them into the first list.
- Repeat the same steps for the second polynomial.

Step 8: Display Input Polynomials

- Display the first polynomial.
- Display the second polynomial.

Step 9: Compute the Sum

- Call the polynomial addition function with the two polynomials.
- Store the returned list as the sum polynomial.

Step 10: Display the Result

- Display the resultant polynomial after addition.

Step 11: Stop

Program:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int coeff;
   int exp;
   struct Node *next;
};

struct Node *createNode(int coeff, int exp) {
   struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
   newNode->coeff = coeff;
   newNode->exp = exp;
   newNode->next = NULL;
   return newNode;
}

struct Node *insertNode(struct Node *head, int coeff, int exp) {
   struct Node *newNode = createNode(coeff, exp);
   if (head == NULL) {
      head = newNode;
   } else {
      struct Node *temp = head;
      while (temp->next != NULL)
         temp = temp->next;
      temp->next = newNode;
   }
   return head;
}

void display(struct Node *head) {
   struct Node *temp = head;
   if (temp == NULL) {
      printf("0\n");
      return;
   }
   while (temp != NULL) {
      printf("%dx^%d", temp->coeff, temp->exp);
      if (temp->next != NULL)
         printf(" + ");
      temp = temp->next;
   }
   printf("\n");
}

struct Node *addPolynomials(struct Node *p1, struct Node *p2) {
   struct Node *result = NULL;
   while (p1 != NULL && p2 != NULL) {
```

```
        if (p1->exp > p2->exp) {
            result = insertNode(result, p1->coeff, p1->exp);
            p1 = p1->next;
        } else if (p2->exp > p1->exp) {
            result = insertNode(result, p2->coeff, p2->exp);
            p2 = p2->next;
        } else {
            int sum = p1->coeff + p2->coeff;
            if (sum != 0)
                result = insertNode(result, sum, p1->exp);
            p1 = p1->next;
            p2 = p2->next;
        }
    }

    while (p1 != NULL) {
        result = insertNode(result, p1->coeff, p1->exp);
        p1 = p1->next;
    }

    while (p2 != NULL) {
        result = insertNode(result, p2->coeff, p2->exp);
        p2 = p2->next;
    }

    return result;
}

void main() {
    struct Node *p1 = NULL, *p2 = NULL, *sum = NULL;
    int n,m, coeff, exp, i;

    printf("Enter number of terms in first polynomial: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Enter coefficient and exponent for term %d: ", i+1);
        scanf("%d%d", &coeff, &exp);
        p1 = insertNode(p1, coeff, exp);
    }

    printf("\nEnter number of terms in second polynomial: ");
    scanf("%d", &m);
    for (i = 0; i < m; i++) {
        printf("Enter coefficient and exponent for term %d: ", i+1);
        scanf("%d%d", &coeff, &exp);
        p2 = insertNode(p2, coeff, exp);
    }

    printf("\nFirst Polynomial: ");
    display(p1);
```

```
    printf("Second Polynomial: ");
    display(p2);

    sum = addPolynomials(p1, p2);

    printf("\nSum of Polynomials: ");
    display(sum);
}
```

Output

```
Enter number of terms in first polynomial: 3
Enter coefficient and exponent for term 1: 2 3
Enter coefficient and exponent for term 2: 5 2
Enter coefficient and exponent for term 3: 3 1

Enter number of terms in second polynomial: 3
Enter coefficient and exponent for term 1: 6 3
Enter coefficient and exponent for term 2: 4 2
Enter coefficient and exponent for term 3: 6 1

First Polynomial: 2x^3 + 5x^2 + 3x^1
Second Polynomial: 6x^3 + 4x^2 + 6x^1

Sum of Polynomials: 8x^3 + 9x^2 + 9x^1
```

b. IMPLEMENT MULTIPLICATION OF POLYNOMIALS USING SINGLY LINKED LISTS.

Aim:

To implement multiplication of polynomials using singly linked lists.

Algorithm:

Step 1: Start

Step 2: Define the Node Structure, Create a structure Node with:

- coeff → coefficient of the term
- exp → exponent of the term
- next → pointer to the next node

  Each node represents one term of a polynomial.

Step 3: Create a New Node

- Allocate memory for a new node.
- Store the given coefficient and exponent.
- Set next to NULL.
- Return the new node.

Step 4: Insert a Term at the End of the Polynomial

- If the polynomial list is empty, make the new node the head.
- Otherwise, traverse to the last node and link the new node at the end.
- Return the updated head pointer.

Step 5: Display a Polynomial

- If the list is empty, print NULL.
- Otherwise, traverse the list:
  - Print each term in the form coeffx^exp.
  - Print " + " between terms where appropriate.
- Move to the next node until the end.

Step 6: Add a Term to the Result Polynomial

- Traverse the result polynomial:
  - If a term with the same exponent exists, add the coefficients and return the result.
- If no matching exponent is found:
  - Create a new node with the given coefficient and exponent.
  - Attach it to the result polynomial.

- Return the updated result polynomial.

Step 7: Multiply Two Polynomials

- Initialize the result polynomial as empty.
- Traverse each term of the first polynomial:
  - For each term, traverse all terms of the second polynomial:
    - Multiply coefficients.
    - Add exponents.
    - Insert the resulting term into the result polynomial using Step 6.
- Continue until all term combinations are processed.
- Return the resultant polynomial.

Step 8: Input Data

- Read the number of terms for the first polynomial.
- Read coefficient and exponent for each term and insert them.
- Repeat the same process for the second polynomial.

Step 9: Display Polynomials

- Display the first polynomial.
- Display the second polynomial.

Step 10: Perform Multiplication

- Call the multiplication function with the two polynomials.
- Store the returned result.

Step 11: Display Result

- Display the resultant polynomial after multiplication.

Step 12: Stop

Program:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int coeff;
   int exp;
   struct Node* next;
};

struct Node* createNode(int coeff, int exp) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->coeff = coeff;
```

```
    newNode->exp = exp;
    newNode->next = NULL;
    return newNode;
}
struct Node* insertEnd(struct Node* head, int coeff, int exp) {
    struct Node* newNode = createNode(coeff, exp);

    if (head == NULL) {
        head = newNode;
    } else {
        struct Node* temp = head;
        while (temp->next != NULL)
            temp = temp->next;
        temp->next = newNode;
    }
    return head;
}

void display(struct Node* head) {
    if (head == NULL) {
        printf("NULL\n");
        return;
    }
    while (head != NULL) {
        printf("%dx^%d", head->coeff, head->exp);
        head = head->next;
        if (head != NULL && head->coeff >= 0)
            printf(" + ");
    }
    printf("\n");
}

struct Node* addTerm(struct Node* result, int coeff, int exp) {
    struct Node* temp = result;
    struct Node* prev = NULL;
    while (temp != NULL) {
        if (temp->exp == exp) {
            temp->coeff += coeff;
            return result;
        }
        prev = temp;
        temp = temp->next;
    }

struct Node* newNode = createNode(coeff, exp);
    if (prev == NULL)
        result = newNode;
    else
        prev->next = newNode;
    return result;
```

```
}

struct Node* multiply(struct Node* poly1, struct Node* poly2) {
    struct Node* result = NULL;
    struct Node* ptr1 = poly1;
    struct Node* ptr2 = poly2;

    while (ptr1 != NULL) {
        ptr2 = poly2;
        while (ptr2 != NULL) {
            int coeff = ptr1->coeff * ptr2->coeff;
            int exp = ptr1->exp + ptr2->exp;
            result = addTerm(result, coeff, exp);
            ptr2 = ptr2->next;
        }
        ptr1 = ptr1->next;
    }
    return result;
}

int main() {
    struct Node* poly1 = NULL;
    struct Node* poly2 = NULL;
    struct Node* result = NULL;
    int n, coeff, exp, i;

    printf("Enter number of terms in first polynomial: ");
    scanf("%d", &n);
    printf("Enter coeff and exp for each term:\n");
    for (i = 0; i < n; i++) {
        scanf("%d%d", &coeff, &exp);
        poly1 = insertEnd(poly1, coeff, exp);
    }

    printf("Enter number of terms in second polynomial: ");
    scanf("%d", &n);
    printf("Enter coeff and exp for each term:\n");
    for (i = 0; i < n; i++) {
        scanf("%d%d", &coeff, &exp);
        poly2 = insertEnd(poly2, coeff, exp);
    }

    printf("\nFirst Polynomial: ");
    display(poly1);
    printf("Second Polynomial: ");
    display(poly2);

    result = multiply(poly1, poly2);

    printf("\nResultant Polynomial after Multiplication: ");
```

```
    display(result);

    return 0;
}
```

Output:

```
Enter number of terms in first polynomial: 3
Enter coeff and exp for each term:
3 3
2 2
1 1
Enter number of terms in second polynomial: 3
Enter coeff and exp for each term:
1 3
1 2
1 1

First Polynomial: 3x^3 + 2x^2 + 1x^1
Second Polynomial: 1x^3 + 1x^2 + 1x^1

Resultant Polynomial after Multiplication: 3x^6 + 5x^5 + 6x^4 + 3x^3 + 1x^2
```

7. CREATE A BINARY TREE FOR A GIVEN SIMPLE ARITHMETIC EXPRESSION AND FIND THE PREFIX / POSTFIX EQUIVALENT.

Aim :

To create a binary tree for a given simple arithmetic expression and find the prefix / postfix Equivalent.

Algorithm:

Step 1: Start

Step 2: Define a tree node structure with:

- data (character)
- left pointer
- right pointer

Also define a stack structure to store tree nodes during tree construction.

Step 3: Initialize the stack to empty (top = NULL).

Step 4: Read the postfix expression from the user.

Step 5: For each symbol in the postfix expression, do the following:

- If the symbol is an operand:
  a. Create a new tree node with the symbol.
  b. Push the node onto the stack.
- If the symbol is an operator:
  a. Pop two nodes from the stack.
  b. Create a new tree node with the operator as data.
  c. Set the first popped node as the right child and the second popped node as the left child.
  d. Push the new operator node back onto the stack.

Step 6: After scanning the entire postfix expression, pop the remaining node from the stack.

This node becomes the root of the expression tree.

Step 7: Display the menu repeatedly with the following options:

1. Create Expression Tree
2. Inorder Traversal (Infix Expression)
3. Preorder Traversal (Prefix Expression)
4. Postorder Traversal (Postfix Expression)
5. Exit

Step 8: Read the user's choice and perform the corresponding operation:

- If choice = 1: Read the postfix expression and create the expression tree using Steps 4–6.
- If choice = 2: If the tree exists, perform inorder traversal:

- Traverse left subtree
- Visit root
- Traverse right subtree

(Use brackets for clarity when visiting operator nodes.)

- If choice = 3: If the tree exists, perform preorder traversal:
  - Visit root
  - Traverse left subtree
  - Traverse right subtree
- If choice = 4: If the tree exists, perform postorder traversal:
  - Traverse left subtree
  - Traverse right subtree
  - Visit root

- If choice = 5: Display exit message and terminate the program.
- Else: Display "Invalid choice".

Step 9: Repeat Step 7 until the user chooses to exit.

Step 10: Stop

Program:

```
#include<stdio.h>
#include<stdlib.h>
#include<ctype.h>

struct Node {
char data;
struct Node *left, *right;
};


struct Stack {
struct Node* data;
struct Stack* next;
};
struct Stack* top = NULL;


struct Node* createNode(char value) {
struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
newNode->data = value;
newNode->left = newNode->right = NULL;
return newNode;
}

void push(struct Node* treeNode) {
struct Stack* temp = (struct Stack*)malloc(sizeof(struct Stack));
temp->data = treeNode;
temp->next = top;
top = temp;
```

```
}
struct Node* pop() {
if (top == NULL) return NULL;
struct Stack* temp = top;
struct Node* res = top->data;

top = top->next;
free(temp);
return res;
}

struct Node* createExpressionTree(char postfix[]) {
int i = 0;
char symbol;
while ((symbol = postfix[i]) != '\0') {
if (isdigit(symbol)) {
struct Node* operandNode = createNode(symbol);
push(operandNode);
} else {
struct Node* operatorNode = createNode(symbol);
operatorNode->right = pop();
operatorNode->left = pop();
push(operatorNode);
}
i++;
}
return pop();
}

void inorder(struct Node* root) {
if (root != NULL) {
if(root->left && root->right) printf("(");
inorder(root->left);
printf("%c", root->data);
inorder(root->right);
if(root->left && root->right) printf(")");
}
}

void preorder(struct Node* root) {
if (root != NULL) {
printf("%c", root->data);
preorder(root->left);
preorder(root->right);
}
}

void postorder(struct Node* root) {
if (root != NULL) {
postorder(root->left);
```

```
postorder(root->right);
printf("%c", root->data);
}
}
int main() {
struct Node* root = NULL;
int choice;
char postfix[100];
do {
printf("\nMenu\n----\n");
printf("1. Create Expression Tree\n");
printf("2. Inorder Traversal (Infix Expression)\n");
printf("3. Preorder Traversal (Prefix Expression)\n");
printf("4. Postorder Traversal (Postfix Expression)\n");
printf("5. Exit\n");
printf("Enter your choice: ");
scanf("%d", &choice);
getchar();
switch(choice) {
case 1:
printf("Enter postfix expression: ");
scanf("%s", postfix);
root = createExpressionTree(postfix);
printf("Expression Tree Created Successfully.\n");
break;
case 2:
if(root == NULL) printf("Create the tree first!\n");
else {
printf("Inorder Traversal (Infix): ");
inorder(root);
printf("\n");
}
break;
case 3:
if(root == NULL) printf("Create the tree first!\n");
else {
printf("Preorder Traversal (Prefix): ");
preorder(root);
printf("\n");
}
break;
case 4:
if(root == NULL) printf("Create the tree first!\n");
else {
printf("Postorder Traversal (Postfix): ");

postorder(root);
printf("\n");
}
```

```
break;
case 5:
printf("Exiting program.\n");
exit(0);
default:
printf("Invalid choice! Try again.\n");
}
} while(1);
return 0;
}
```

Output:

```
Menu
----
1. Create Expression Tree
2. Inorder Traversal (Infix Expression)
3. Preorder Traversal (Prefix Expression)
4. Postorder Traversal (Postfix Expression)
5. Exit
Enter your choice: 1
Enter postfix expression: 53+62/*35*+
Expression Tree Created Successfully.
Menu
----
1. Create Expression Tree
2. Inorder Traversal (Infix Expression)
3. Preorder Traversal (Prefix Expression)
4. Postorder Traversal (Postfix Expression)
5. Exit
Enter your choice: 2
Inorder Traversal (Infix): (((5+3)*(6/2))+(3*5))
Menu
----
1. Create Expression Tree
2. Inorder Traversal (Infix Expression)
3. Preorder Traversal (Prefix Expression)
4. Postorder Traversal (Postfix Expression)
5. Exit
Enter your choice: 3
Preorder Traversal (Prefix): +*+53/62*35

Menu
----
1. Create Expression Tree
2. Inorder Traversal (Infix Expression)
3. Preorder Traversal (Prefix Expression)
4. Postorder Traversal (Postfix Expression)
5. Exit
Enter your choice: 4
```

Postorder Traversal (Postfix): 53+62/*35*+
Menu
---
1. Create Expression Tree
2. Inorder Traversal (Infix Expression)
3. Preorder Traversal (Prefix Expression)
4. Postorder Traversal (Postfix Expression)
5. Exit
Enter your choice: 5
Exiting program.

8. IMPLEMENT A DICTIONARY OF WORD-MEANING PAIRS USING BINARY SEARCH TREES.

Aim:

To implement a dictionary of word-meaning pairs using binary search trees.

Algorithm:

Step 1: Start

Step 2: Define Structure

- Define a structure Node with:
  - word to store the word
  - meaning to store the meaning
  - Pointers left and right for left and right child nodes

Step 3: Create Node

1. Allocate memory for a new node
2. Copy the given word and meaning into the node
3. Set left and right pointers to NULL
4. Return the new node

Step 4: Insert Word

1. If the tree is empty (root == NULL):
   - Create a new node and return it as root
2. If the new word is alphabetically smaller than the root word:
   - Insert it into the left subtree
3. If the new word is alphabetically greater than the root word:
   - Insert it into the right subtree
4. If the word already exists:
   - Display a message that the word already exists
5. Return the root

Step 5: Search Word

1. If the tree is empty:
   - Display "Word not found!"
2. If the word matches the root word:
   - Display the meaning
3. If the word is alphabetically smaller:
   - Search in the left subtree
4. If the word is alphabetically greater:
   - Search in the right subtree

Step 6: Display Dictionary (In-order Traversal)

1. If the current node is not NULL:
    - Traverse the left subtree
    - Display the word and its meaning
    - Traverse the right subtree
2. This displays all words in alphabetical order

Step 7: Main Menu

1. Display menu options:
    - Insert Word
    - Search Word
    - Display Dictionary
    - Exit
2. Read the user's choice
3. Perform the selected operation using switch
4. Repeat until the user selects Exit

Step 8: Stop

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Node {
   char word[50];
   char meaning[100];
   struct Node *left, *right;
};

struct Node* createNode(char word[], char meaning[]) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   strcpy(newNode->word, word);
   strcpy(newNode->meaning, meaning);
   newNode->left = newNode->right = NULL;
   return newNode;
}

struct Node* insert(struct Node* root, char word[], char meaning[]) {
   if (root == NULL)
      return createNode(word, meaning);

   if (strcmp(word, root->word) < 0)
      root->left = insert(root->left, word, meaning);
   else if (strcmp(word, root->word) > 0)
      root->right = insert(root->right, word, meaning);
   else
      printf("Word already exists in dictionary.\n");
```

```
    return root;
}

void search(struct Node* root, char word[]) {
    if (root == NULL) {
        printf("Word not found!\n");
        return;
    }

    if (strcmp(word, root->word) == 0)
        printf("Meaning of '%s' is: %s\n", word, root->meaning);
    else if (strcmp(word, root->word) < 0)
        search(root->left, word);
    else
        search(root->right, word);
}

void inorder(struct Node* root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%s : %s\n", root->word, root->meaning);
        inorder(root->right);
    }
}

int main() {
    struct Node* root = NULL;
    int choice;
    char word[50], meaning[100];

    do {
        printf("\n--- Dictionary Menu ---\n");
        printf("1. Insert Word\n2. Search Word\n3. Display Dictionary\n4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        getchar();

        switch (choice) {
            case 1:
                printf("Enter Word: ");
                gets(word);
                printf("Enter Meaning: ");
                gets(meaning);
                root = insert(root, word, meaning);
                break;
            case 2:
                printf("Enter Word to Search: ");
                gets(word);
                search(root, word);
                break;
```

```
            case 3:
                printf("\n--- Dictionary Words ---\n");
                inorder(root);
                break;
            case 4:
                printf("Exiting...\n");
                exit(0);
            default:
                printf("Invalid choice. Try again.\n");
        }
    } while (1);

    return 0;
}
```

Output:

```
--- Dictionary Menu ---
1. Insert Word
2. Search Word
3. Display Dictionary
4. Exit
Enter your choice: 1
Enter Word: Apple
Enter Meaning: A fruit

--- Dictionary Menu ---
1. Insert Word
2. Search Word
3. Display Dictionary
4. Exit
Enter your choice: 1
Enter Word: Cat
Enter Meaning: A domestic animal

--- Dictionary Menu ---
1. Insert Word
2. Search Word
3. Display Dictionary
4. Exit
Enter your choice: 1
Enter Word: Elephant
Enter Meaning: A wild animal

--- Dictionary Menu ---
1. Insert Word
2. Search Word
3. Display Dictionary
4. Exit
Enter your choice: 2
```

```
Enter Word to Search: Cat
Meaning of 'Cat' is: A domestic animal

--- Dictionary Menu ---
1. Insert Word
2. Search Word
3. Display Dictionary
4. Exit
Enter your choice: 2
Enter Word to Search: dog
Word not found!

--- Dictionary Menu ---
1. Insert Word
2. Search Word
3. Display Dictionary
4. Exit
Enter your choice: 3

--- Dictionary Words ---
Apple : A fruit
Cat : A domestic animal
Elephant : A wild animal

--- Dictionary Menu ---
1. Insert Word
2. Search Word
3. Display Dictionary
4. Exit
Enter your choice: 4
Exiting...
```

9. LAND MINES INSIDE MAZE

Aim:

Find the shortest distance of every cell from a landmine inside a maze.

Algorithm:

Step 1: Start

Begin the execution of the program.

Step 2: Declare Variables

- Declare an integer array queue[100] to implement the queue.
- Declare two integers front and rear and initialize them to -1.
- Declare an integer array visited[100] and initialize all elements to 0.
- Declare variables to store:
    - Number of vertices n
    - Adjacency matrix adj[10][10]
    - Starting vertex start

Step 3: Read Input

1. Read the number of vertices n.
2. Read the adjacency matrix adj[n][n]:
    - adj[i][j] = 1 indicates an edge between vertex i and j
    - adj[i][j] = 0 indicates no edge
3. Read the starting vertex start.

Step 4: Enqueue Starting Vertex

1. Insert the starting vertex into the queue using enqueue(start).
2. Mark the starting vertex as visited by setting visited[start] = 1.
3. Print the message "BFS Traversal:".

Step 5: Perform BFS Traversal

Repeat the following steps while the queue is not empty (front <= rear):

1. Remove the front element from the queue using dequeue() and store it in current.
2. Print the value of current.
3. For every vertex i from 0 to n-1:
    - If adj[current][i] == 1 and visited[i] == 0:
        - Insert vertex i into the queue.
        - Mark vertex i as visited.

Step 6: Continue Until Queue Is Empty

- Continue visiting and enqueuing adjacent unvisited vertices level by level.
- BFS ensures that all vertices at the current level are visited before moving to

the next level.

Step 7: Stop

Program:

```
#include<stdio.h>
#include<stdlib.h>

#define MAX 100
#define INF 9999

int dirRow[] = {-1, 1, 0, 0};
int dirCol[] = {0, 0, -1, 1};

struct Node {
   int x, y;
};

struct Node queue[MAX*MAX];
int front = 0, rear = -1;

void enqueue(int x, int y) {
   rear++;
   queue[rear].x = x;
   queue[rear].y = y;
}

struct Node dequeue() {
   return queue[front++];
}

int isValid(int x, int y, int n, int m) {
   return (x >= 0 && x < n && y >= 0 && y < m);
}

int main() {
   int maze[MAX][MAX], dist[MAX][MAX];
   int n, m;

   printf("Enter number of rows and columns: ");
   scanf("%d%d", &n, &m);

   printf("Enter the maze (-1 for mine, 0 for path, # for wall as 999):\n");
   for (int i=0; i<n; i++)
      for (int j=0; j<m; j++)
         scanf("%d", &maze[i][j]);

   for (int i=0; i<n; i++)
      for (int j=0; j<m; j++)
```

```
            dist[i][j] = INF;

    for (int i=0; i<n; i++)
        for (int j=0; j<m; j++) {
            if (maze[i][j] == -1) {
                enqueue(i, j);
                dist[i][j] = 0;
            }
        }

    while (front <= rear) {
        struct Node temp = dequeue();
        int x = temp.x;
        int y = temp.y;

        for (int k=0; k<4; k++) {
            int newX = x + dirRow[k];
            int newY = y + dirCol[k];

            if (isValid(newX, newY, n, m) && maze[newX][newY] == 0 &&
dist[newX][newY] > dist[x][y] + 1) {
                dist[newX][newY] = dist[x][y] + 1;
                enqueue(newX, newY);
            }
        }
    }

    printf("Shortest distance from land mines:\n");
    for (int i=0; i<n; i++) {
        for (int j=0; j<m; j++) {
            if (maze[i][j] == 999) printf("  # ");
            else if (dist[i][j] == INF) printf(" INF ");
            else printf("%3d ", dist[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Output:

```
Enter number of rows and columns: 5 5
Enter the maze (-1 for mine, 0 for path, # for wall as 999):
0 0 0 0 0
0 -1 0 0 0
0 0 0 0 -1
0 0 0 0 0
-1 0 0 0 999
```

Shortest distance from land mines:

```
2 1 2 3 2
1 0 1 2 1
2 1 2 1 0
1 2 3 2 1
0 1 2 3 #
```

10. WATER POURING PROBLEM - GRAPH MODELLING

Aim:

We have three containers whose sizes are 10 litres, 7 litres, and 4 litres, respectively. The 7-litre and 4-litre containers start out full of water, but the 10-litre container is initially empty. We are allowed one type of operation: pouring the contents of one container into another, stopping only when the source container is empty or the destination container is full. We want to know if there is a sequence of pourings that leaves exactly 2 litres in the 7 or 4-litre container. Model this as a graph problem and solve.

Algorithm:

Step 1: Start

Step 2: Define a structure called State to store the amount of water in jugs A, B, and C along with a pointer to its parent state.

Step 3: Define a Queue data structure to perform Breadth-First Search (BFS).

Step 4: Initialize the queue to be empty.

Step 5: Create a three-dimensional array visited[a][b][c] to keep track of already visited states and initialize all values to zero.

Step 6: Create the initial state with water levels (A = 0, B = 7, C = 4).

Step 7: Insert the initial state into the queue and mark it as visited.

Step 8: Repeat the following steps while the queue is not empty:

- a. Remove (dequeue) the front state from the queue.
- b. Check if jug B contains 2 liters or jug C contains 2 liters.
  - If yes, print the solution path by tracing back through parent states and stop the program.
- c. Generate all possible next states by pouring water between jugs:
  - Pour from A to B
  - Pour from A to C
  - Pour from B to A
  - Pour from B to C
  - Pour from C to A
  - Pour from C to B
- d. For each pouring operation:
  - Calculate the maximum possible water that can be transferred.
  - Create a new state after pouring.
  - If the new state is not visited, mark it as visited and insert it into the queue.

- If the queue becomes empty and no solution is found, display "No solution found".

Step 9: Stop

Program:

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 11
typedef struct State {
   int a, b, c;
   struct State* parent;
} State;

typedef struct QueueNode {
   State *state;
   struct QueueNode *next;
} QueueNode;

typedef struct Queue {
   QueueNode *front, *rear;
} Queue;

State* createState(int a, int b, int c, State *parent) {
   State* newState = (State*)malloc(sizeof(State));
   newState->a = a;
   newState->b = b;
   newState->c = c;
   newState->parent = parent;
   return newState;
}

void initQueue(Queue *q) {
   q->front = q->rear = NULL;
}

int isEmpty(Queue *q) {
   return q->front == NULL;
}

void enqueue(Queue *q, State *state) {
   QueueNode *temp = (QueueNode*)malloc(sizeof(QueueNode));
   temp->state = state;
   temp->next = NULL;
   if (q->rear == NULL) {
      q->front = q->rear = temp;
      return;
   }
   q->rear->next = temp;
   q->rear = temp;
}

State* dequeue(Queue *q) {
```

```
    if (q->front == NULL)
        return NULL;
    QueueNode *temp = q->front;
    State *state = temp->state;
    q->front = q->front->next;
    if (q->front == NULL)
        q->rear = NULL;
    free(temp);
    return state;
}

void printPath(State *state) {
    if (state == NULL)
        return;
    printPath(state->parent);
    printf("(%d, %d, %d)\n", state->a, state->b, state->c);
}

void pour(State *curr, Queue *q, int visited[MAX][MAX][MAX]) {
    int A = curr->a, B = curr->b, C = curr->c;
    int maxA = 10, maxB = 7, maxC = 4;

    int pourAmt = (B + A > maxB) ? maxB - B : A;
    if (pourAmt > 0) {
        int na = A - pourAmt, nb = B + pourAmt, nc = C;
        if (!visited[na][nb][nc]) {
            visited[na][nb][nc] = 1;
            enqueue(q, createState(na, nb, nc, curr));
        }
    }

    pourAmt = (C + A > maxC) ? maxC - C : A;
    if (pourAmt > 0) {
        int na = A - pourAmt, nb = B, nc = C + pourAmt;
        if (!visited[na][nb][nc]) {
            visited[na][nb][nc] = 1;
            enqueue(q, createState(na, nb, nc, curr));
        }
    }

    pourAmt = (A + B > maxA) ? maxA - A : B;
    if (pourAmt > 0) {
        int na = A + pourAmt, nb = B - pourAmt, nc = C;
        if (!visited[na][nb][nc]) {
            visited[na][nb][nc] = 1;
            enqueue(q, createState(na, nb, nc, curr));
        }
    }

    pourAmt = (C + B > maxC) ? maxC - C : B;
```

```
    if (pourAmt > 0) {
        int na = A, nb = B - pourAmt, nc = C + pourAmt;
        if (!visited[na][nb][nc]) {
            visited[na][nb][nc] = 1;
            enqueue(q, createState(na, nb, nc, curr));
        }
    }

    pourAmt = (A + C > maxA) ? maxA - A : C;
    if (pourAmt > 0) {
        int na = A + pourAmt, nb = B, nc = C - pourAmt;
        if (!visited[na][nb][nc]) {
            visited[na][nb][nc] = 1;
            enqueue(q, createState(na, nb, nc, curr));
        }
    }

    pourAmt = (B + C > maxB) ? maxB - B : C;
    if (pourAmt > 0) {
        int na = A, nb = B + pourAmt, nc = C - pourAmt;
        if (!visited[na][nb][nc]) {
            visited[na][nb][nc] = 1;
            enqueue(q, createState(na, nb, nc, curr));
        }
    }
}

int main() {
    Queue q;
    initQueue(&q);

    int visited[MAX][MAX][MAX] = {0};

    State *start = createState(0, 7, 4, NULL);
    enqueue(&q, start);
    visited[0][7][4] = 1;

    while (!isEmpty(&q)) {
        State *curr = dequeue(&q);

        if (curr->b == 2 || curr->c == 2) {
            printf("\nSolution path:\n");
            printPath(curr);
            return 0;
        }

        pour(curr, &q, visited);
    }

    printf("No solution found.\n");
```

```
    return 0;
}
```

Output:

```
Solution path:
(0, 7, 4)
(4, 7, 0)
(10, 1, 0)
(6, 1, 4)
(6, 5, 0)
(2, 5, 4)
(2, 7, 2)
```

11. IMPLEMENT THE FIND AND REPLACE FEATURE IN A TEXT EDITOR.

Aim:

Implement the find and replace feature in a text editor.

Algorithm:

Step 1: Start

Step 2: Input

1. Read the input text string
2. Read the word to be found
3. Read the replacement word

Step 3: Initialize

- Set index i = 0 for scanning the text
- Set index j = 0 for storing the result text

Step 4: Find and Replace Process

1. Repeat until the end of the text string is reached
2. Assume the word is found (found = 1)
3. Compare the substring of text starting at position i with the word to be found
4. If any character does not match:
    - Set found = 0
5. If the word is found:
    - Copy the replacement word into the result string
    - Move index i forward by the length of the word to be found
6. If the word is not found:
    - Copy the current character from text to result
    - Move index i forward by one position

Step 5: Terminate Result

- Add null character ('\0') at the end of the result string

Step 6: Output

- Display the modified text

Step 7: Stop

Program:

```
#include <stdio.h>
#include <string.h>

int main() {
```

```
    char text[500], find[50], replace[50];
    char result[500];
    int i = 0, j = 0, k, found;

    printf("Enter the text:\n");
    gets(text);

    printf("Enter word to find: ");
    gets(find);

    printf("Enter word to replace: ");
    gets(replace);

    while (text[i] != '\0') {
        found = 1;
        k = 0;

        while (find[k] != '\0') {
            if (text[i + k] != find[k]) {
                found = 0;
                break;
            }
            k++;
        }

        if (found) {
            k = 0;
            while (replace[k] != '\0') {
                result[j++] = replace[k++];
            }
            i += strlen(find);
        } else {
            result[j++] = text[i++];
        }
    }
    result[j] = '\0';

    printf("\nModified Text:\n%s\n", result);

    return 0;
}
```

Output:

```
Enter the text:
I love programming
Enter word to find: love
Enter word to replace: enjoy

Modified Text:
I enjoy programming
```

12. BINARY SEARCH

Aim:

Given an array of sorted items, implement an efficient algorithm to search for specific items in the array.

Algorithm:

Step 1: Start

Step 2: Input

1. Read the number of elements n
2. Read n sorted elements into array arr
3. Read the element item to be searched

Step 3: Initialize

- Set beg = 0
- Set end = n − 1
- Compute mid = (beg + end) / 2

Step 4: Binary Search Process

1. Repeat while beg ≤ end and arr[mid] ≠ item
2. If arr[mid] < item
    - Set beg = mid + 1
3. Else
    - Set end = mid − 1
4. Recalculate mid = (beg + end) / 2

Step 5: Check Result

- If arr[mid] == item
    - Display “Element found” and its position (mid + 1)
- Else
    - Display “Element not found”

Step 6: Stop

Program:

```
#include <stdio.h>
void main()
{
int arr[100], n, i, item;
int beg, end, mid;

printf("Enter number of elements: ");
```

```
scanf("%d", &n);
printf("Enter %d sorted elements:\n", n);
for(i = 0; i < n; i++) {
scanf("%d", &arr[i]);
}

printf("Enter the element to search: ");
scanf("%d", &item);

beg = 0;
end = n - 1;

while((beg <= end) && (arr[mid]!=item))
{
if(arr[mid] < item)
beg = mid + 1;
else
end = mid - 1;
mid = (beg+end)/2;
}
if(arr[mid] == item)
printf("Element %d found at position %d.\n", item, mid + 1);
else
printf("Element %d not found in the array.\n", item);
}
```

Output:

```
Enter number of elements: 5
Enter 5 sorted elements:
2 4 7 9 12
Enter the element to search: 2 4 7 9 12
Element 2 found at position 1.
Enter number of elements: 5
Enter 5 sorted elements:
2 3 4 6 8
Enter the element to search: 9
Element 9 not found in the array.
```

13. IMPLEMENT BUBBLE SORT, INSERTION SORT, RADIX SORT, QUICK SORT, AND MERGE SORT AND COMPARE THE NUMBER OF STEPS INVOLVED.

a. BUBBLE SORT

Aim:

Implement bubble sort.

Algorithm:

Step 1: Start

Step 2: Input

1. Read the number of elements n
2. Read n array elements into array arr

Step 3: Display Original Array

- Print all elements of the array

Step 4: Bubble Sort Process

1. Repeat for i = 0 to n − 2
2. Repeat for j = 0 to n − i − 2
3. Compare adjacent elements arr[j] and arr[j + 1]
4. If arr[j] > arr[j + 1]:
    - Swap the two elements
5. After each pass, the largest unsorted element moves to its correct position

Step 5: Display Sorted Array

- Print all elements of the sorted array

Step 6: Stop

Program:

```
#include <stdio.h>
void bubbleSort(int arr[], int n)
{
  int i, j, temp;
  for (i = 0; i < n-1; i++)
  {
     for (j = 0; j < n-i-1; j++)
     {
       if (arr[j] > arr[j+1])
       {
            temp = arr[j];
            arr[j] = arr[j+1];
```

```
                arr[j+1] = temp;
            }
        }
    }
}

void printArray(int arr[], int n)
{
    for (int i=0; i<n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}
void readArray(int arr[], int n)
{
    for (int i=0; i<n; i++)
        scanf("%d", &arr[i]);
}
int main()
{
    int arr[50] ;
    int n;

    printf("Enter the number of elements \n");
    scanf("%d",&n);
    printf("Enter the array elements\n");
    readArray(arr,n);

    printf("Original array: ");
    printArray(arr, n);

    bubbleSort(arr, n);

    printf("Sorted array: ");
    printArray(arr, n);

    return 0;
}
```

Output:

```
Enter the number of elements
5
Enter the array elements
65 36 2 15 9
Original array: 65 36 2 15 9
Sorted array: 2 9 15 36 65
```

b. INSERTION SORT

Aim:

Implement Insertion Sort.

Algorithm:

Step 1: Start

Step 2: Input

1. Read the number of elements n
2. Read n elements into array arr

Step 3: Display Original Array

- Print all elements of the original array

Step 4: Insertion Sort Process

1. Assume the first element of the array is already sorted
2. For each element from index 1 to n − 1:
    1. Store the current element in key
    2. Compare key with elements before it
    3. Shift all elements greater than key one position to the right
    4. Insert key into its correct position
    5. Display the array after each pass

Step 5: Display Sorted Array

- Print all elements of the sorted array

Step 6: Stop

Program:

```
#include <stdio.h>
void printArray(int arr[], int n)
{
   for (int i=0; i<n; i++)
      printf("%d ", arr[i]);
   printf("\n");
}
void readArray(int arr[], int n)
{
   for (int i=0; i<n; i++)
      scanf("%d", &arr[i]);

}

void insertionSort(int arr[], int n)
```

```
{
  int i,j,key;
   for(i = 1; i < n; i++)
   {
      key = arr[i];
      j = i - 1;

      while(j >= 0 && arr[j] > key)
      {
         arr[j + 1] = arr[j];
         j--;
      }

      arr[j + 1] = key;
      printArray(arr, n);
   }
}

int main()
{
   int arr[50] ;
   int n;

   printf("Enter the number of elements \n");
   scanf("%d",&n);
   printf("Enter the array elements\n");
   readArray(arr,n);

   printf("Original array: ");
   printArray(arr, n);

   insertionSort(arr, n);

   printf("Sorted array: ");
   printArray(arr, n);

   return 0;
}
```

Output:

```
Enter the number of elements
5
Enter the array elements
9 1 8 6 5
Original array: 9 1 8 6 5
1 9 8 6 5
1 8 9 6 5
1 6 8 9 5
1 5 6 8 9
Sorted array: 1 5 6 8 9
```

c. RADIX SORT

Aim:

Implement radix sort.

Algorithm:

Step 1: Start

Step 2: Read the number of elements

- Read integer n (number of elements in the array).

Step 3: Read array elements

- Read n integers into array arr[] using readArray().

Step 4: Display the original array

- Print all elements of arr[] using printArray().

Step 5: Find the maximum element

- Use getMax(arr, n) to find the largest element in the array.
- Store the maximum value in variable max.

Step 6: Perform Radix Sort

- Initialize exp = 1 (represents digit position: units, tens, hundreds, etc.).
- Repeat the following steps while max / exp > 0:

a) Counting Sort based on digit at exp

1. Create an output array output[] of size n.
2. Create a count array count[10] and initialize all elements to 0.
3. Count the occurrences of each digit:
    - Extract digit using (arr[i] / exp) % 10.
    - Increment the corresponding count index.
4. Modify the count array to store cumulative counts.
5. Build the output array by placing elements in correct positions.
6. Copy sorted elements from output[] back to arr[].
7. Print the array after each digit sorting.

b) Move to next digit

- Multiply exp by 10.

Step 7: Display the sorted array

- Print the final sorted array using printArray().

Step 8: Stop

Program:

```
#include<stdio.h>
void printArray(int arr[], int n) {
   for (int i=0; i<n; i++)
      printf("%d ", arr[i]);
   printf("\n");
}
void readArray(int arr[], int n) {
   for (int i=0; i<n; i++)
      scanf("%d", &arr[i]);

}
int getMax(int arr[], int n) {
   int max = arr[0];
   for (int i = 1; i < n; i++)
      if (arr[i] > max)
         max = arr[i];
   return max;
}

void countingSort(int arr[], int n, int exp)
{
   int output[n];
   int count[10] = {0};

   for (int i = 0; i < n; i++)
      count[(arr[i] / exp) % 10]++;

   for (int i = 1; i < 10; i++)
      count[i] += count[i - 1];

   for (int i = n - 1; i >= 0; i--)
   {
      int digit = (arr[i] / exp) % 10;
      output[count[digit] - 1] = arr[i];
      count[digit]--;
   }

   for (int i = 0; i < n; i++)
      arr[i] = output[i];
      printArray(arr, n);
}

void radixSort(int arr[], int n)
{
   int max = getMax(arr, n);
   for (int exp = 1; max / exp > 0; exp *= 10)
```

```
        countingSort(arr, n, exp);
}

int main()
{
   int arr[50] ;
   int n;

   printf("Enter the number of elements \n");
   scanf("%d",&n);
   printf("Enter the array elements\n");
   readArray(arr,n);

   printf("Original array: ");
   printArray(arr, n);

   radixSort(arr,n);

   printf("Sorted array: ");
   printArray(arr, n);

   return 0;
}
```

Output:

```
Enter the number of elements
5
Enter the array elements
89 52 98 22 13
Original array: 89 52 98 22 13
52 22 13 98 89
13 22 52 89 98
Sorted array: 13 22 52 89 98
```

d. QUICK SORT

Aim:

Implement Quick Sort.

Algorithm:

Step 1: Start

Step 2: Read number of elements

- Read integer n (size of the array).

Step 3: Read array elements

- Read n integers into array arr[] using readArray().

Step 4: Display original array

- Print the elements of the array from index 0 to n−1 using printArray().

Step 5: Apply Quick Sort

- Call quicksort(arr, 0, n−1).

Step 6: Quick Sort Procedure

If low < high, perform the following steps:

a) Partition the array

1. Choose the first element (arr[low]) as the pivot.
2. Initialize:
   - i = low + 1
   - j = high
3. Move i from left to right until an element greater than the pivot is found.
4. Move j from right to left until an element less than or equal to the pivot is found.
5. If i < j, swap arr[i] and arr[j].
6. Repeat steps 3–5 until i ≥ j.
7. Swap the pivot element with arr[j].
8. Return the pivot position j.

b) Display partitioned array

- Print elements before the pivot.
  Print the pivot element marked with *.
- Print elements after the pivot.

c) Recursive calls

- Recursively apply Quick Sort to:
  - Left sub-array: low to pivotIndex − 1
  - Right sub-array: pivotIndex + 1 to high

Step 7: Display sorted array

- After all recursive calls, print the fully sorted array.

Step 8: Stop

Program:

```
#include <stdio.h>
void printArray(int arr[],int low, int high) {
   for (int i=low; i<high; i++)
```

```
        printf("%d ", arr[i]);

}
void readArray(int arr[], int n) {
   for (int i=0; i<n; i++)
      scanf("%d", &arr[i]);

}


void swap(int *a, int *b) {
   int temp = *a;
   *a = *b;
   *b = temp;
}


int partition(int arr[], int low, int high) {
   int pivot = arr[low];
   int i = low + 1;
   int j = high;

   while (1) {
         while (i <= high && arr[i] <= pivot) {
          i++;
      }

      while (j >= low && arr[j] > pivot) {
         j--;
      }

      if (i < j) {
         swap(&arr[i], &arr[j]);
      } else {
         break;
      }
   }

   swap(&arr[low], &arr[j]);

   return j;
}


void quicksort(int arr[], int low, int high) {
   if (low < high) {
      int pivotIndex = partition(arr, low, high);
      printArray(arr,low,pivotIndex);
      printf("\t*%d\t",arr[pivotIndex]);
      printArray(arr,pivotIndex+1,high+1);
```

```
        printf("\n");
        quicksort(arr, low, pivotIndex - 1);
        quicksort(arr, pivotIndex + 1, high);
    }
}

int main() {
    int arr[50] ;
    int n;

    printf("Enter the number of elements \n");
    scanf("%d",&n);
    printf("Enter the array elements\n");
    readArray(arr,n);

    printf("Original array: ");
    printArray(arr,0, n);
    printf("\n");
    quicksort(arr,0,n-1);

    printf("Sorted array: ");
    printArray(arr,0, n);
    printf("\n");
    return 0;
}
```

Output:

```
Enter the number of elements
5
Enter the array elements
66 45 25 8 1
Original array: 66 45 25 8 1
1 45 25 8       *66
        *1      45 25 8
8 25    *45
        *8      25
Sorted array: 1 8 25 45 66
```

e. MERGE SORT

Aim:

Implement Quick Sort.

Algorithm:

Step 1: Start the program.

Step 2: Read the total number of elements in the array.

Step 3: Read all the array elements from the user.

Step 4: Display the original array.

Step 5: Check whether the array has more than one element.

Step 6: Find the middle position of the array.

Step 7: Divide the array into two sub-arrays: left sub-array and right sub-array.

Step 8: Recursively apply merge sort on the left sub-array.

Step 9: Recursively apply merge sort on the right sub-array.

Step 10: Create two temporary arrays to store elements of the left and right sub-arrays.

Step 11: Compare elements from both sub-arrays one by one.

Step 12: Copy the smaller element into the original array.

Step 13: Continue comparison until one of the sub-arrays is exhausted.

Step 14: Copy the remaining elements of the left sub-array, if any, into the original array.

Step 15: Copy the remaining elements of the right sub-array, if any, into the original array.

Step 16: Repeat the merging process until all sub-arrays are merged.

Step 17: The array becomes completely sorted.

Step 18: Display the sorted array.

Step 19: Stop

Program:

```
#include <stdio.h>
void merge(int arr[], int left, int mid, int right) {
    int i, j, k;
    int n1 = mid - left + 1;
    int n2 = right - mid;
    int L[n1], R[n2];
    for (i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    i = 0;
    j = 0;
    k = left;
```

```
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        }
        else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }

    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }
}

void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        merge(arr, left, mid, right);
    }
}

void printArray(int arr[], int n) {
    for (int i=0; i<n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}
void readArray(int arr[], int n) {
    for (int i=0; i<n; i++)
        scanf("%d", &arr[i]);

}
int main() {
    int arr[50] ;
    int n;
```

```
    printf("Enter the number of elements \n");
    scanf("%d",&n);
    printf("Enter the array elements\n");
    readArray(arr,n);

    printf("Original array: ");
    printArray(arr, n);

    mergeSort(arr,0,n-1);

    printf("Sorted array: ");
    printArray(arr, n);

    return 0;
}
```

Output:

```
Enter the number of elements
5
Enter the array elements
12 20 5 56 8
Original array: 12 20 5 56 8
Sorted array: 5 8 12 20 56
```

14. CUSTOMER PRIORITY QUEUE USING LINKED LIST

Aim:

The General post office wishes to give preferential treatment to its customers. They have identified the customer categories as Defence personnel, Differently abled, Senior citizen, Ordinary. The customers are to be given preference in the decreasing order - Differently abled, Senior citizen, Defence personnel, Normal person. Generate the possible sequence of completion.

Algorithm:

Step 1: Start

Step 2: Declare a structure Node with

- name (customer name)
- priority (customer priority)
- next pointer to the next node

Step 3: Initialize front pointer to NULL.

Step 4: Display the menu:

1. Add Customer
2. Serve Customer
3. Display Queue
4. Exit

Step 5: Read the user's choice.

Step 6: If choice = 1 (Add Customer):

1. Read customer name and priority.
2. Create a new node.
3. If queue is empty or new priority is greater than front priority:
    - Insert the node at the front.
4. Else:
    - Traverse the queue to find correct position.
    - Insert the node in descending order of priority.
5. Display updated queue.

Step 7: If choice = 2 (Serve Customer):

1. Check if queue is empty.
2. If empty, display message.
3. Else:
    - Remove the front node.
    - Display served customer details.
    - Free the removed node.
4. Display updated queue.

Step 8: If choice = 3 (Display Queue):

1. Traverse the queue from front to end.
2. Display customer name and priority.

Step 9: If choice = 4 (Exit):

1. Terminate the program.

Step 10: Repeat Steps 4–9 until choice = 4.

Step 11: Stop

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Node {
   char name[100];
   int priority;
   struct Node* next;
};

struct Node* front = NULL;

struct Node* createNode(char name[], int priority) {
   struct Node* temp = (struct Node*)malloc(sizeof(struct Node));
   strcpy(temp->name, name);
   temp->priority = priority;
   temp->next = NULL;
   return temp;
}

void enqueue(char name[], int priority) {
   struct Node* newNode = createNode(name, priority);

   if (front == NULL || priority > front->priority) {
      newNode->next = front;
      front = newNode;
   } else {
      struct Node* temp = front;
      while (temp->next != NULL && temp->next->priority >= priority) {
         temp = temp->next;
      }
      newNode->next = temp->next;
      temp->next = newNode;
   }
```

```
    printf("Customer %s added with priority %d\n", name, priority);
}

void dequeue() {
    if (front == NULL) {
        printf("No customers to serve.\n");
    } else {
        struct Node* temp = front;
        printf("Serving Customer: %s (Priority: %d)\n", front->name, front->priority);
        front = front->next;
        free(temp);
    }
}

void displayQueue() {
    struct Node* temp = front;
    if (temp == NULL) {
        printf("No customers in queue.\n");
    } else {
        printf("\nQueue:\n");
        while (temp != NULL) {
            printf("%s (Priority: %d) -> ", temp->name, temp->priority);
            temp = temp->next;
        }
        printf("NULL\n");
    }
}


int main() {
    int choice;
    char name[100], category[100];
    int priority;

    do {
        printf("\nMenu\n----\n");
        printf("1. Add Customer\n");
        printf("2. Serve Customer\n");
        printf("3. Display Queue\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        getchar();

        switch(choice) {
            case 1:
                printf("Enter customer name: ");
                gets(name);
                printf("Enter customer category(Differently Abled-4/Senior
Citizen-3/Defence Personnel-2/Ordinary-1): ");
```

```
            scanf("%d",&priority);
            enqueue(name, priority);
            displayQueue();
            break;
        case 2:
            dequeue();
            displayQueue();
            break;
        case 3:
            displayQueue();
            break;
        case 4:
            printf("Exiting...\n");
            break;
        default:
            printf("Invalid choice.\n");
      }
   } while(choice != 4);

   return 0;
}
```

Output:

```
Menu
----
1. Add Customer
2. Serve Customer
3. Display Queue
4. Exit
Enter your choice: 1
Enter customer name: Rajamma
Enter customer category(Differently Abled-4/Senior Citizen-3/Defence
Personnel-2/Ordinary-1): 3
Customer Rajamma added with priority 3

Queue:
Rajamma (Priority: 3) -> NULL

Menu
----
1. Add Customer
2. Serve Customer
3. Display Queue
4. Exit
Enter your choice: 1
Enter customer name: Ram
Enter customer category(Differently Abled-4/Senior Citizen-3/Defence
Personnel-2/Ordinary-1): 1
Customer Ram added with priority 1
```

```
Queue:
Rajamma (Priority: 3) -> Ram (Priority: 1) -> NULL

Menu
----
1. Add Customer
2. Serve Customer
3. Display Queue
4. Exit
Enter your choice: 1
Enter customer name: Raman
Enter customer category(Differently Abled-4/Senior Citizen-3/Defence
Personnel-2/Ordinary-1): 4
Customer Raman added with priority 4

Queue:
Raman (Priority: 4) -> Rajamma (Priority: 3) -> Ram (Priority: 1) -> NULL

Menu
----
1. Add Customer
2. Serve Customer
3. Display Queue
4. Exit
Enter your choice: 1
Enter customer name: Ajitha
Enter customer category(Differently Abled-4/Senior Citizen-3/Defence
Personnel-2/Ordinary-1): 2
Customer Ajitha added with priority 2

Queue:
Raman (Priority: 4) -> Rajamma (Priority: 3) -> Ajitha (Priority: 2) -> Ram (Priority:
1) -> NULL

Menu
----
1. Add Customer
2. Serve Customer
3. Display Queue
4. Exit
Enter your choice: 3

Queue:
Raman (Priority: 4) -> Rajamma (Priority: 3) -> Ajitha (Priority: 2) -> Ram (Priority:
1) -> NULL

Menu
----
1. Add Customer
```

```
2. Serve Customer
3. Display Queue
4. Exit
Enter your choice: 2
Serving Customer: Raman (Priority: 4)

Queue:
Rajamma (Priority: 3) -> Ajitha (Priority: 2) -> Ram (Priority: 1) -> NULL

Menu
----
1. Add Customer
2. Serve Customer
3. Display Queue
4. Exit
Enter your choice: 2
Serving Customer: Rajamma (Priority: 3)

Queue:
Ajitha (Priority: 2) -> Ram (Priority: 1) -> NULL

Menu
----
1. Add Customer
2. Serve Customer
3. Display Queue
4. Exit
Enter your choice: 3

Queue:
Ajitha (Priority: 2) -> Ram (Priority: 1) -> NULL

Menu
----
1. Add Customer
2. Serve Customer
3. Display Queue
4. Exit
Enter your choice: 4
Exiting...
```

15. DICTIONARY IMPLEMENTATION WITH SUGGESTIONS

Aim:

Implement a spell checker using a hash table to store a dictionary of words for fast lookup. Implement functions to check if a word is valid and to suggest corrections for misspelled words.

Algorithm:

Step 1: Start

Step 2: Define a hash table of fixed size using an array of pointers, where each index stores a Step 3: linked list to handle collisions using chaining.

Step 4: Define a hash function to convert a word into a valid hash table index.

Step 5: Initialize the hash table to be empty.

Step 6: Insert all dictionary words into the hash table:

- For each word, compute its hash value.
- Create a new node and store the word in it.
- Insert the node at the beginning of the linked list at the computed index.

Step 7: Read a word from the user for spell checking.

Step 8: Search the word in the hash table:

- Compute the hash value of the word.
- Traverse the linked list at the corresponding index.
- If the word is found, display that it is spelled correctly and stop.

Step 9: If the word is not found in the dictionary:

- Display that the word is not in the dictionary.
- Generate spelling suggestions by considering all words with edit distance one:
  - a. Replace each character of the word with letters from ‘a’ to ‘z’ (substitution).
  - b. Insert letters from ‘a’ to ‘z’ at every position in the word (insertion).
  - c. Remove one character at a time from the word (deletion).
- For each generated word, search it in the hash table.
- Display all valid words found as suggestions.

Step 10: Stop

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
```

```
#define TABLE_SIZE 1000
#define WORD_LEN 50


typedef struct Node {
   char word[WORD_LEN];
   struct Node* next;
} Node;

Node* hashTable[TABLE_SIZE];

unsigned int hash(const char* str) {
   unsigned long hash = 5381;
   int c;
   while ((c = *str++))
      hash = hash*33 + c;
   return hash % TABLE_SIZE;
}

void insert(const char* word) {
   unsigned int index = hash(word);
   Node* newNode = (Node*)malloc(sizeof(Node));
   strcpy(newNode->word, word);
   newNode->next = hashTable[index];
   hashTable[index] = newNode;
}

int search(const char* word) {
   unsigned int index = hash(word);
   Node* curr = hashTable[index];
   while (curr) {
      if (strcmp(curr->word, word) == 0)
         return 1;
      curr = curr->next;
   }
   return 0;
}

void suggest(const char* word) {
   char temp[WORD_LEN];
   int len = strlen(word);

   printf("Did you mean:\n");

   for (int i = 0; i < len; i++) {
      for (char c = 'a'; c <= 'z'; c++) {
         strcpy(temp, word);
         temp[i] = c;
         if (search(temp) && strcmp(temp, word) != 0)
```

```
                printf("  %s\n", temp);
            }
        }

        for (int i = 0; i <= len; i++) {
            for (char c = 'a'; c <= 'z'; c++) {
                strcpy(temp, word);
                memmove(temp + i + 1, temp + i, len - i + 1);
                temp[i] = c;
                if (search(temp))
                    printf("  %s\n", temp);
            }
        }

        for (int i = 0; i < len; i++) {
            strcpy(temp, word);
            memmove(temp + i, temp + i + 1, len - i);
            if (search(temp))
                printf("  %s\n", temp);
        }
    }

    int main() {
        char* dictionary[] = {"apple", "apply", "bat", "ball", "cat", "dog",
    "cart","dog","elephant"};
        int dictSize = sizeof(dictionary) / sizeof(dictionary[0]);

        for (int i = 0; i < dictSize; i++) {
            insert(dictionary[i]);
        }

        char word[WORD_LEN];
        printf("Enter a word to check: ");
        scanf("%s", word);

        if (search(word)) {
            printf("'%s' is spelled correctly.\n", word);
        } else {
            printf("'%s' is NOT in dictionary.\n", word);
            suggest(word);
        }
        return 0;
    }
```

Output:

```
Enter a word to check: elepant
'elepant' is NOT in dictionary.
Did you mean:
  elephant
```

16. GARBAGE COLLECTION AND COMPACTION

Aim:

Simulation of a basic memory allocator and garbage collector using doubly linked list.

Algorithm:

Step 1: Start

Step 2: Define a structure Block containing:

- start_address
- size
- is_free (to indicate free or used block)
- Pointers prev and next for doubly linked list.

Step 3: Initialize the head of the doubly linked list as NULL.

Step 4: Create a function to create a new memory block with given start address, size, and status.

Step 5: Insert memory blocks into the doubly linked list in sequence.

Step 6: Display the memory blocks by traversing the linked list and showing start address, size, and status.

Step 7: Perform Garbage Collection:

- Traverse the linked list.
- If two consecutive blocks are free, merge them into a single block.
- Update links and free the merged block.
- Continue until no adjacent free blocks remain.

Step 8: Display the memory blocks after garbage collection.

Step 9: Perform Compaction:

- Traverse the list and collect all used blocks.
- Reassign contiguous memory addresses starting from the base address.
- Create a new list containing only used blocks in order.
- Calculate total free memory size.
- Create one large free block at the end using the total free size.

Step 10: Delete the old memory list and update the head to the new compacted list.

Step 11: Display the memory blocks after compaction.

Step 12: Stop

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Block {
   int start_address;
   int size;
   int is_free;
   struct Block* prev;
   struct Block* next;
};

struct Block* head = NULL;

struct Block* create_block(int start, int size, int is_free) {
   struct Block* new_block = (struct Block*)malloc(sizeof(struct Block));
   new_block->start_address = start;
   new_block->size = size;
   new_block->is_free = is_free;
   new_block->next = NULL;
   new_block->prev = NULL;
   return new_block;
}

void add_block(int start, int size, int is_free) {
   struct Block* new_block = create_block(start, size, is_free);

   if (head == NULL) {
      head = new_block;
      return;
   }

   struct Block* temp = head;
   while (temp->next != NULL) {
      temp = temp->next;
   }

   temp->next = new_block;
   new_block->prev = temp;
}

void display_memory() {
   struct Block* temp = head;
   printf("\nMemory Blocks:\n");
   printf("----------------------------\n");
   while (temp != NULL) {
      printf("Start: %d, Size: %d, Status: %s\n",
            temp->start_address,
```

```
            temp->size,
            temp->is_free ? "Free" : "Used");
        temp = temp->next;
    }
    printf("----------------------------\n");
}

void garbage_collection() {
    struct Block* temp = head;

    while (temp != NULL && temp->next != NULL) {
        if (temp->is_free && temp->next->is_free) {

            temp->size += temp->next->size;
            struct Block* to_delete = temp->next;
            temp->next = to_delete->next;

            if (to_delete->next != NULL) {
                to_delete->next->prev = temp;
            }

            free(to_delete);
        } else {
            temp = temp->next;
        }
    }
}

void compaction() {
    struct Block* new_head = NULL;
    struct Block* last_used = NULL;
    int current_address = 1000;

    struct Block* temp = head;
    while (temp != NULL) {
        if (!temp->is_free) {
            struct Block* new_block = create_block(current_address, temp->size, 0);
            current_address += temp->size;

            if (new_head == NULL) {
                new_head = new_block;
                last_used = new_head;
            } else {
                last_used->next = new_block;
                new_block->prev = last_used;
                last_used = new_block;
            }
        }
        temp = temp->next;
    }
```

```
    int free_size = 0;
    temp = head;
    while (temp != NULL) {
        if (temp->is_free) {
            free_size += temp->size;
        }
        temp = temp->next;
    }

    if (free_size > 0) {
        struct Block* free_block = create_block(current_address, free_size, 1);
        if (new_head == NULL) {
            new_head = free_block;
        } else {
            last_used->next = free_block;
            free_block->prev = last_used;
        }
    }

    temp = head;
    while (temp != NULL) {
        struct Block* next = temp->next;
        free(temp);
        temp = next;
    }

    head = new_head;
}

int main() {

    add_block(1000, 200, 0);
    add_block(1200, 150, 1);
    add_block(1350, 300, 0);
    add_block(1650, 100, 1);
    add_block(1750, 200, 1);

    printf("\nBefore Garbage Collection and Compaction:");
    display_memory();

    garbage_collection();
    printf("\nAfter Garbage Collection:");
    display_memory();

    compaction();
    printf("\nAfter Compaction:");
    display_memory();

    return 0;
```

```
}
```

Output:

```
Before Garbage Collection and Compaction:
Memory Blocks:
----------------------------
Start: 1000, Size: 200, Status: Used
Start: 1200, Size: 150, Status: Free
Start: 1350, Size: 300, Status: Used
Start: 1650, Size: 100, Status: Free
Start: 1750, Size: 200, Status: Free
----------------------------

After Garbage Collection:
Memory Blocks:
----------------------------
Start: 1000, Size: 200, Status: Used
Start: 1200, Size: 150, Status: Free
Start: 1350, Size: 300, Status: Used
Start: 1650, Size: 300, Status: Free
----------------------------

After Compaction:
Memory Blocks:
----------------------------
Start: 1000, Size: 200, Status: Used
Start: 1200, Size: 300, Status: Used
Start: 1500, Size: 450, Status: Free
----------------------------
```

17. KTU MAX ACTIVITY POINTS

Aim:

The CSE dept is organizing a tech fest with so many exciting events. By participating in an event, you can claim for activity points as stipulated by KTU. Each event i gives you A[i] activity points where A is an array. If you are not allowed to participate in more than k events, what's the max number of points that you can earn?

Algorithm:

Step 1: Start

Step 2: Read the number of events n.

Step 3: Read the activity points of all n events into an array A.

Step 4: Read the maximum number of events k that can be participated in.

Step 5: Sort the array A in descending order using Bubble Sort:

- Compare adjacent elements.
- Swap them if the earlier element is smaller than the next.
- Repeat the process until the array is sorted in descending order.

Step 6: Initialize maxPoints to 0.

Step 7: Add the activity points of the first k elements (or up to n elements if k > n) from the sorted array.

Step 8: Display the value of maxPoints as the maximum activity points.

Step 9: Stop

Program:

```
#include <stdio.h>
void sortDescending(int arr[], int n) {
   for (int i = 0; i < n - 1; i++) {
      for (int j = 0; j < n - i - 1; j++) {
         if (arr[j] < arr[j + 1]) {
            int temp = arr[j];
            arr[j] = arr[j + 1];
            arr[j + 1] = temp;
         }
      }
   }
}

int main() {
```

```
    int n, k;

    printf("Enter number of events: ");
    scanf("%d", &n);

    int A[n];
    printf("Enter activity points for each event:\n");
    for (int i = 0; i < n; i++) {
       scanf("%d", &A[i]);
    }

    printf("Enter maximum events you can participate in (k): ");
    scanf("%d", &k);

    sortDescending(A, n);

    int maxPoints = 0;
    for (int i = 0; i < k && i < n; i++) {
       maxPoints += A[i];
    }

    printf("Maximum Activity Points: %d\n", maxPoints);

    return 0;
}
```

Output:

```
Enter number of events: 5
Enter activity points for each event:
5
2
8
9
10
Enter maximum events you can participate in (k): 2
Maximum Activity Points: 19
```

18. MERGE K SORTED LINKED LISTS USING HEAP

Aim:

Merge K sorted lists into a single sorted list using a heap. Use a min-heap to keep track of the smallest element from each list. Repeatedly extract the smallest element and insert the next element from the corresponding list into the heap until all lists are merged.

Algorithm:

Step 1: Start

Step 2: Define a structure for a linked list node with two fields: data and next.

Step 3: Define a structure for a heap node containing a pointer to a linked list node and the index of the list.

Step 4: Read the number of linked lists k and create an array of k linked list heads.

Step 5: Insert the first node of each non-empty linked list into the min-heap.

Step 6: Build a min-heap based on the data values of the nodes.

Step 7: Create a dummy node and initialize a pointer tail to build the merged linked list.

Step 8: While the heap is not empty, do the following:

- Remove the minimum element from the heap.
- Attach this node to the merged linked list.
- Move the tail pointer to the next position.
- If the removed node has a next node, insert it into the heap.
- Otherwise, reduce the heap size.
- Restore the min-heap property using heapify.

Step 9: Set the head of the merged linked list as dummy.next.

Step 10: Display the merged sorted linked list.

Step 11: Stop

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>

struct Node {
   int data;
   struct Node* next;
};
```

```
struct Node* newNode(int data) {
    struct Node* temp = (struct Node*)malloc(sizeof(struct Node));
    temp->data = data;
    temp->next = NULL;
    return temp;
}

struct HeapNode {
    struct Node* node;
    int listIndex;
};

void swap(struct HeapNode* a, struct HeapNode* b) {
    struct HeapNode temp = *a;
    *a = *b;
    *b = temp;
}

void heapify(struct HeapNode heap[], int n, int i) {
    int smallest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && heap[left].node->data < heap[smallest].node->data)
        smallest = left;

    if (right < n && heap[right].node->data < heap[smallest].node->data)
        smallest = right;

    if (smallest != i) {
        swap(&heap[i], &heap[smallest]);
        heapify(heap, n, smallest);
    }
}

void buildHeap(struct HeapNode heap[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(heap, n, i);
}

struct Node* mergeKLists(struct Node* arr[], int k) {
    struct HeapNode* heap = (struct HeapNode*)malloc(sizeof(struct HeapNode) * k);
    int heapSize = 0;

    for (int i = 0; i < k; i++) {
        if (arr[i] != NULL) {
            heap[heapSize].node = arr[i];
            heap[heapSize].listIndex = i;
            heapSize++;
        }
```

```
    }
    buildHeap(heap, heapSize);

    struct Node dummy;
    struct Node* tail = &dummy;
    dummy.next = NULL;

    while (heapSize > 0) {
        struct HeapNode root = heap[0];

        tail->next = root.node;
        tail = tail->next;

        if (root.node->next != NULL) {
            heap[0].node = root.node->next;
        } else {
            heap[0] = heap[heapSize - 1];
            heapSize--;
        }

        heapify(heap, heapSize, 0);
    }

    return dummy.next;
}

// Display linked list
void printList(struct Node* node) {
    while (node != NULL) {
        printf("%d--> ", node->data);
        node = node->next;
    }
    printf("NULL\n");
}

// Example usage
int main() {
    int k = 3; // number of linked lists

    struct Node* arr[k];

    // List 1: 1 -> 4 -> 5
    arr[0] = newNode(1);
    arr[0]->next = newNode(4);
    arr[0]->next->next = newNode(5);
    arr[0]->next->next->next=NULL;
    printf("List1\n");
    printList(arr[0]);
    // List 2: 1 -> 3 -> 4
    arr[1] = newNode(1);
```

```
    arr[1]->next = newNode(3);
    arr[1]->next->next = newNode(4);
    arr[1]->next->next->next=NULL;
    printf("List2\n");
    printList(arr[1]);
    // List 3: 2 -> 6
    arr[2] = newNode(2);
    arr[2]->next = newNode(6);
    arr[2]->next->next=NULL;
    printf("List3\n");
    printList(arr[2]);
    struct Node* result = mergeKLists(arr, k);

    printf("Merged Sorted Linked List:\n");
    printList(result);

    return 0;
}
```

Output:

```
List1
1--> 4--> 5--> NULL
List2
1--> 3--> 4--> NULL
List3
2--> 6--> NULL
Merged Sorted Linked List:
1--> 1--> 2--> 3--> 4--> 4--> 5--> 6--> NULL
```